University of Plymouth

School of Engineering, Computing, and Mathematics

COMP3000

Computing Project

2025/2026

Stronghold

Mark Hardy

10857365

BSc (Hons) Computer Science (Cyber Security)

Acknowledgements

I would like to thank my supervisor, Bogdan Ghita, for his continuous support throughout the development of this project and providing valuable feedback for improvements.

Abstract

Most web browsers available on the market today are helpful at mitigating the risk of children encountering malicious content whilst browsing the internet. However, none of the available options are useful at teaching why content could be dangerous.

Stronghold is a security-focussed web browser, aimed at teaching children safe internet browsing behaviours. The intention of Stronghold is to act as a fun, educational tool which parents can rely on to keep their children safe whilst allowing them to learn about the dangers of the internet. Gamification aspects, such as a levelling system and challenges, are used to reward and encourage safe browsing behaviours like identifying potentially malicious site links and download types, while penalising users who attempt to access them.

Stronghold will act as a transitional web browser, where children can begin their journey on the internet while mitigating their risk of encountering malicious content in an enjoyable yet informative fashion.

Table of Contents

Word Count

Words – 10,518

Code Link

GitHub – https://github.com/Mark7567/Stronghold

1 Introduction

This report will discuss the creation of a security-focussed web browser titled ‘Stronghold’. The purpose of this project is to teach children how to browse the internet safely, utilising behavioural conditioning and gamification aspects to do so. The project was initially targeted towards the elderly due to their high presence as victims of malicious attacks such as phishing[1], however the decision to implement gamification led to a pivot to a target audience of young children who may benefit more from learning about internet safety[2] before using less safe, unrestricted internet browsing. Leaning towards behavioural conditioning then felt to be the best pivot since children may be more responsive when using positive reinforcement and structured feedback techniques[3]. The project will finalise in a browser that aims to reduce risks and encourage safer browsing behaviour.

The report will cover the initial design that had been planned for implementation, including wireframes, storyboards, flowcharts, and use cases for each initial page and feature, the methodology behind the development lifecycle, the main features that were both planned and implemented in the final design, the unique selling points that make the browser different from other products, the testing that was performed on the initial designs, including how testing was performed and how the results of testing influenced future decisions, the issues and challenges that could potentially have been faced throughout development, as well as those that were actually faced, and a final conclusion discussing the overall thoughts and feelings surrounding the project as a whole.

2 Background, Objectives, and Deliverables

2.1 Background

As children gain access to the internet from younger ages[4], it is important to ensure that they have knowledge of potential threats they may face, while also keeping them safe when online. The increase in cyber threats such as phishing[5] heighten the importance of learning safe browsing practices, especially while young and therefore susceptible to behavioural changes[6].

Traditional browsers such as Google Chrome or Microsoft Edge are acceptable solutions for keeping high security standards and allowing children to remain safe while browsing online; however, these solutions do not teach users why their behaviour is unsafe, meaning they are susceptible to performing it repeatedly and developing unsafe browsing behaviour. Warnings are often also passive and can be easily ignored or misunderstood by users. As a result, a solution is needed to not only protect users from online threats, but to help them understand why a website or a download may be unsafe.

Stronghold aims to act as the solution by utilising behavioural conditioning to establish safe browsing habits within children while ensuring their safety online. A risk score is calculated for each website a user attempts to access, combining a multitude of factors such as domain structure analysis and typosquatting prevention to determine whether a site is safe to access. By combining constructive feedback with a punishment and reward system, this aims to improve both user safety and awareness of threats when online.

2.2 Literature Analysis

2.2.1 Online Threats

The modern internet contains a wide range of threats that should be considered when determining user safety. The 2024 Data Breach Investigation Report by Verizon[7] highlighted that the most common attack vector was exploiting credentials, with the most common access point being web applications. The report also identifies additional attack vectors including phishing, denial of service, and exploiting vulnerabilities, demonstrating the vast number of threats that a user may face while online and therefore need to be prepared for.

Malware remains a significant risk to internet users, particularly through compromised websites and unsafe downloads. Guidance from the National Cyber Security Centre[8] details steps that can be taken to mitigate the risk of a malware infection; although this relates primarily to organisations, guidance equally applies to individual users. The guidance highlights the importance of having correct malware protection in place to reduce the likelihood of an infection occurring by blocking access to downloads onto the device.

Many threats posed rely heavily on manipulation and user deception rather than purely technical methods. Research from Zscaler (2024)[9] highlighted the dangers of typosquatting and brand impersonation to manipulate users into accessing malicious domains. Typosquatting is often combined with other malicious methods such as phishing, credential theft, or malware delivery to increase the likelihood of a successful attack. The research also discovered that over half of phishing domains utilised free TLS certificates which in turn allowed them to bypass traditional browser security. This demonstrates that indicators such as possession of a TLS certificate are not a sufficient way to determine the safety of a website as it can enhance credibility of sites possessing one.

Additionally, the research observed that typosquatting domains could be used to initiate an automatic download of malicious payloads, such as trojans. This highlights the importance of considering file-based threats alongside domain-based threats when deciding security features.

2.2.2 User Behaviour

Research has revealed that users are not always responsive to security warnings that appear on browsers when dangerous actions are being performed. A study performed by Sunshine et al. (2009)[10] found that many users portrayed dangerous behaviour by ignoring warnings, both that were premade by browsers such as Firefox and Internet Explorer, and warnings created by the conductors of this study. The conclusion of this study revealed that blocking users from unsafe websites and eliminating warnings would be a more suitable method. This study is limited however by its age, as it contains details and uses warnings present in older internet applications, which are no longer present. This study did contain many participants, with 100 participants being involved, which supports statements made in the study; however, the participants involved were all adults which may not correctly reflect browsing habits within children. Despite these limitations, the study highlights an important issue: users may not reliably respond to warnings and require access from malicious sites being blocked. Stronghold aims to tackle this by utilising warnings on medium risk sites and blocking access completely to high-risk sites.

A significantly larger study containing over 6,000 participants was conducted by Reeder et al. (2018)[11]. This study identified that while a proportion of users do

adhere to warnings, a significant number ignore them or fail to comprehend their meaning. A lack of comprehension influenced user decisions more than habituation, although this played a small role. This suggests that even when warnings are adhered to, user knowledge is not increased and there is no improvement in understanding the risk. This is vital for younger users who may lack the knowledge to understand security messages; this is something Stronghold aims to resolve through gamification and specialty quizzes, as well as messages alongside the warnings.

Further research conducted by Vance et al. (2017)*[12]* revealed that users habituated to warnings over a five-day workweek, making them more likely to ignore the warnings as they habituated. The study compared both static and polymorphic warnings, with the study concluding that polymorphic warnings improved users' responses to important security messages. This study is also limited by its age slightly, however due to its focus being on habituation, this does not apply as harshly as studies primarily focussing on the internet. Additionally, this research involved only 16 participants, all of whom were adults within a work environment. Findings from this study do however support the idea that repeated exposure reduces the effectiveness of warnings, which presents a challenge to systems that rely on warnings. Stronghold was able to adapt from this by utilising a mixture of blocking and warning to encourage learning and reduce the likelihood of warnings becoming ignored.

This concept is additionally supported by a study conducted by Kirwan et al. (2020)*[13]* who used an MRI scanner to examine the neurological responses to repeated exposure. The study found that repeated exposure reduced stimulus processing and attention, indicating that users become desensitised and fatigued to constant exposure. While the overlap in authors compared to the previous study limit the available perspectives, the use of neuroscientific methods strengthens the validity of the findings. This suggests that how users process warnings may extend beyond conscious decisions due to the existence of fatigue, which has influenced Stronghold to alter the warning pages by stating the reasons for why the warning has appeared, alongside adjusting between URL warnings and download warnings.

2.2.3 Inexperienced Users

A study conducted by Ofcom (2024)[14] revealed that a vast majority of children use technological devices, with finding online images and playing games being some of the most common activities performed. This reveals that children are getting involved in online environments from a young age and therefore are exposing themselves to potential risks due to their inexperience.

While the study highlights the increase in internet usage among children, it fails to identify that younger users may not have the experience to keep themselves safe while online by verifying domains and avoiding malicious downloads for example. This creates a greater reliance on external systems to allow for online safety while navigating online.

Furthermore, the study found that parents recognise that the internet is a learning space for their children, suggesting that online environments should allow for both safety and education. This reinforces the idea that only blocking unsafe content may not be suitable for developing long-term behaviours, and there is instead a need for systems that encourage learning while developing safer browsing habits.

2.2.4 Existing Products

Modern existing products such as Google Chrome[15], Microsoft Edge[16], Mozilla Firefox[17], and Opera[18] often include built-in security mechanisms to prevent or restrict access to malicious sites and downloads by utilising phishing protection, malware protection, and blacklisting sites flagged as dangerous. Tools such as Google Safe Browsing and Microsoft Defender SmartScreen are used to perform these tasks, reducing the likelihood of a user accessing threatening content.

While protection provided by these systems is known to be effective, they do not assist in improving understanding of threats faced. Users are typically blocked from accessing their content but receive a limited explanation as to why, causing a reliance on blocking without understanding how to improve browsing behaviours.

Furthermore, these systems often rely on databases of known threats instead of analysing a domain's characteristics when the user attempts to access it. This may limit their ability to block new threats or more complicated techniques such as typosquatting.

This highlights a gap in the market where a solution is required to enforce learning techniques while providing a safe system. Combining education with security allows users to remain safe while understanding what they are being kept safe from.

2.3 SMART Objectives

The primary objectives of this project were defined using the SMART framework to ensure they were specific, measurable, achievable, relevant, and time bound. The objectives are outlined as the following:

- To design and develop a functional, security-focussed web browser using the Electron framework which allows for basic navigation and searching of both URLs and queries within 5 seconds of opening the browser
- To take appropriate action of either warning against or blocking suspicious URLs based on a risk scoring system and depending on the user's security settings within 2 seconds of the URL being searched or selected from a Google search query to prevent access to the site and instead display the relevant page explaining reasons for the action
- To implement a tiered download prevention system which intercepts and blocks or warns against downloads with commonly malicious extensions within 1 second of the download being processed to ensure malicious downloads do not get installed on user devices
- To implement a dashboard page which is linked to Firebase and displays a user's current level, experience points, blocked URLs, blocked downloads, ignored warnings, and recent changes, which update within 5 seconds of a relevant user action being completed.
- To create a daily quiz containing 1 question that a signed-in user can access and complete within 2 minutes to receive a reward of 10 experience points if the correct answer is chosen
- To create a weekly quiz containing 10 questions that a signed-in user can access and complete within 5 minutes to receive a reward of 5 experience points per correct answer, with a total of 50 experience points being available

- To implement Google OAUTH sign in connected to Firebase which allows a user to sign in with their Google account and view their statistics and settings within 3 minutes

2.4 Deliverables

The final deliverables expected from this project are:

- A working prototype for an Electron-based browser
- This 10,000-word report documenting the creation of this project
- A 5-minute video to demonstrate the features of this project
- A GitHub repository containing necessary code for the project to function
- A non-technical poster to explain the purpose of this project

3 Design

This section showcases the design decisions made throughout the development of this project. It works through both UI design and feature design, detailing the initial wireframes and flowcharts used to plan each page, how the final implementation compared to these designs, and features and unique selling points which were planned, explaining why each feature was deemed necessary, how it was intended to be implemented, and whether it was included in the final project or not.

3.1 Wireframes

The initial wireframes were designed using Figma[19] to create an understanding of button layout, colour options, and basic user flow which is expanded on with flowcharts. Due to the target audience for this project being children, the designs focussed on simplicity and providing an easy understanding of how to use the browser. Each wireframe acted as a visual plan to receive feedback on before the final design was implemented. Several elements of each page were added, changed, or removed based on this user feedback and constraints with development.

3.1.1 Startup Page

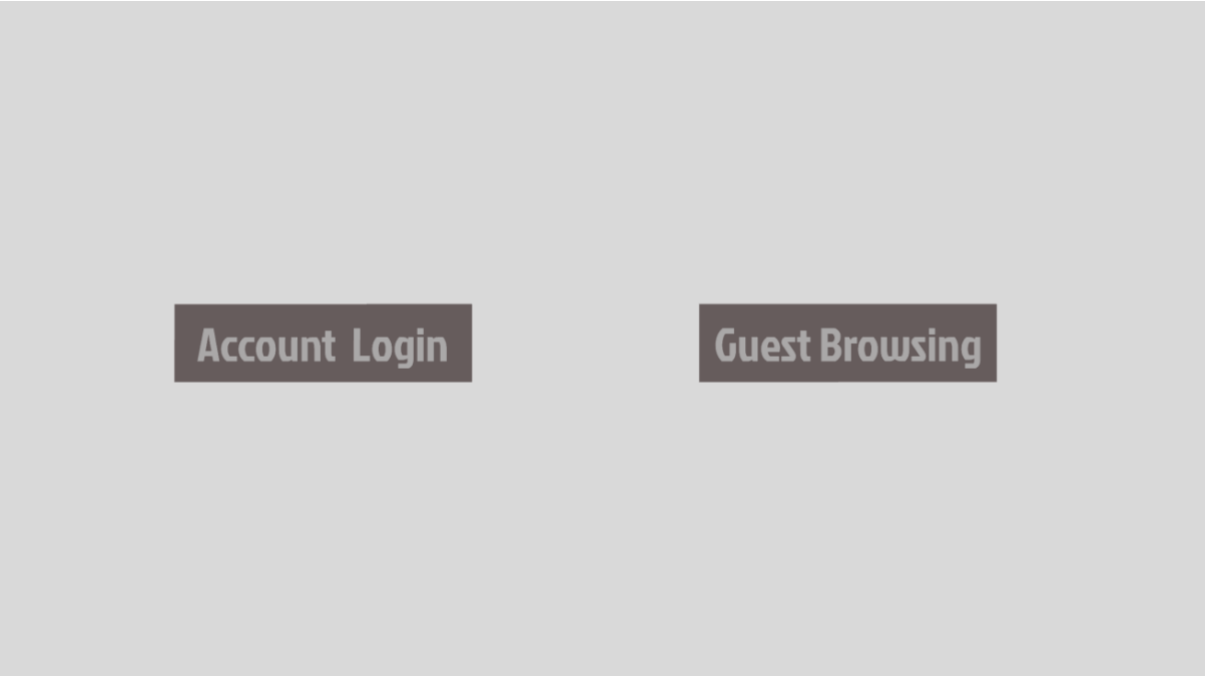

Figure 1 – Wireframe, Stronghold Startup Page

Figure 1 shows the wireframe for the startup page. This page was designed to be the starting point for the browser, providing the user with the option to sign into an account where they could track security statistics and customise their experience, or browse as a guest with the strictest security options enabled and no gamification. The initial design was simple, containing only a Google sign-in button and a guest browsing button.

3.1.2 Home Page

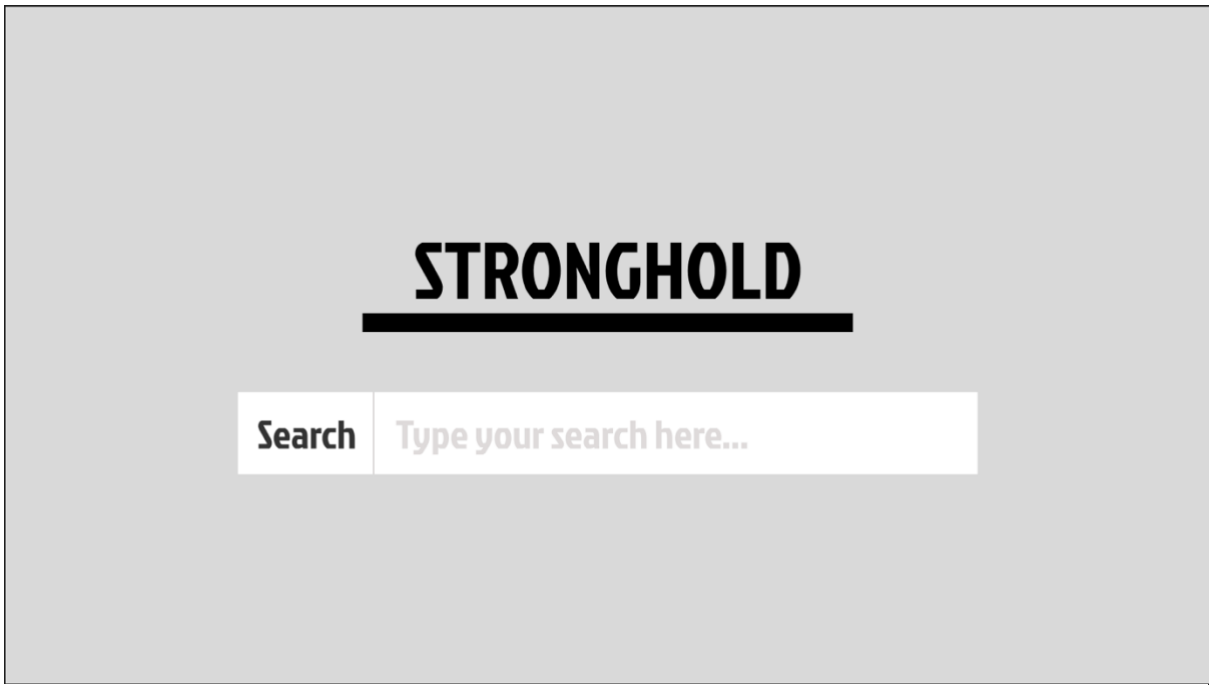

Figure 2 – Wireframe, Stronghold Home Page

Figure 2 shows the wireframe for the home page. This page was designed to be the central point that the user would see after signing in or when a new tab was opened. The design was kept simple, providing the user only with the option to enter a URL or search query into the search box.

3.1.3 Dashboard

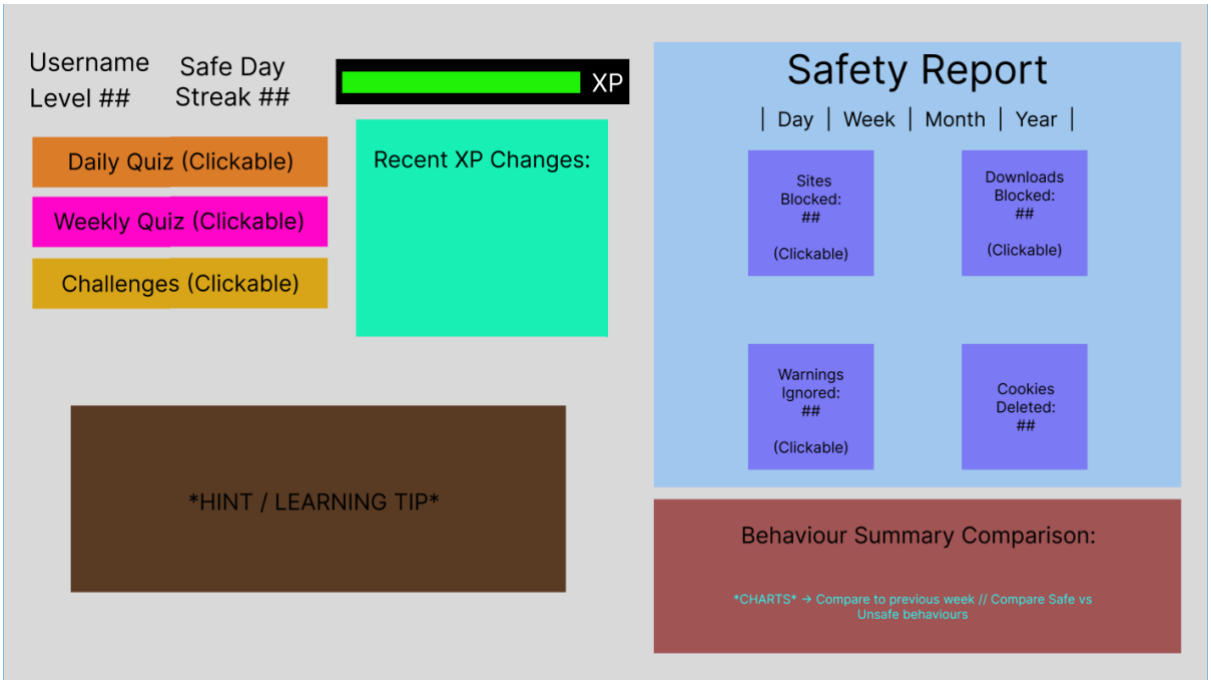

Figure 3 – Wireframe, Stronghold Dashboard

Figure 3 shows the wireframe created for the dashboard. This page was designed to be the centralised point for all gamification elements of the browser, containing features such as a safety report for browsing which tracked blocked sites, blocked downloads, warnings ignored, and cookies deleted. The dashboard wireframe also contained a behaviour summary, hints and tips, quizzes and challenges, and experience point logic such as the levelling system and recent changes to experience. This was a much more complicated design as this was the page intended to encourage the behavioural conditioning and learning elements of the browser.

3.1.4 Taskbar

Figure 4 – Wireframe, Stronghold Taskbar

Figure 4 shows the wireframe created for the taskbar. This page was designed to be constantly displayed, allowing the user to enter a URL or search query into the search bar, access other relevant pages such as the home page, dashboard, and

settings page, display recent downloads, allow pages to be bookmarked and displayed, and act as a general form of navigation for the browser. The initial design was slightly complicated due to the limited space, requiring a small addition to the top of each page, and the number of features needing to be implemented in that small space.

3.1.5 Settings Page

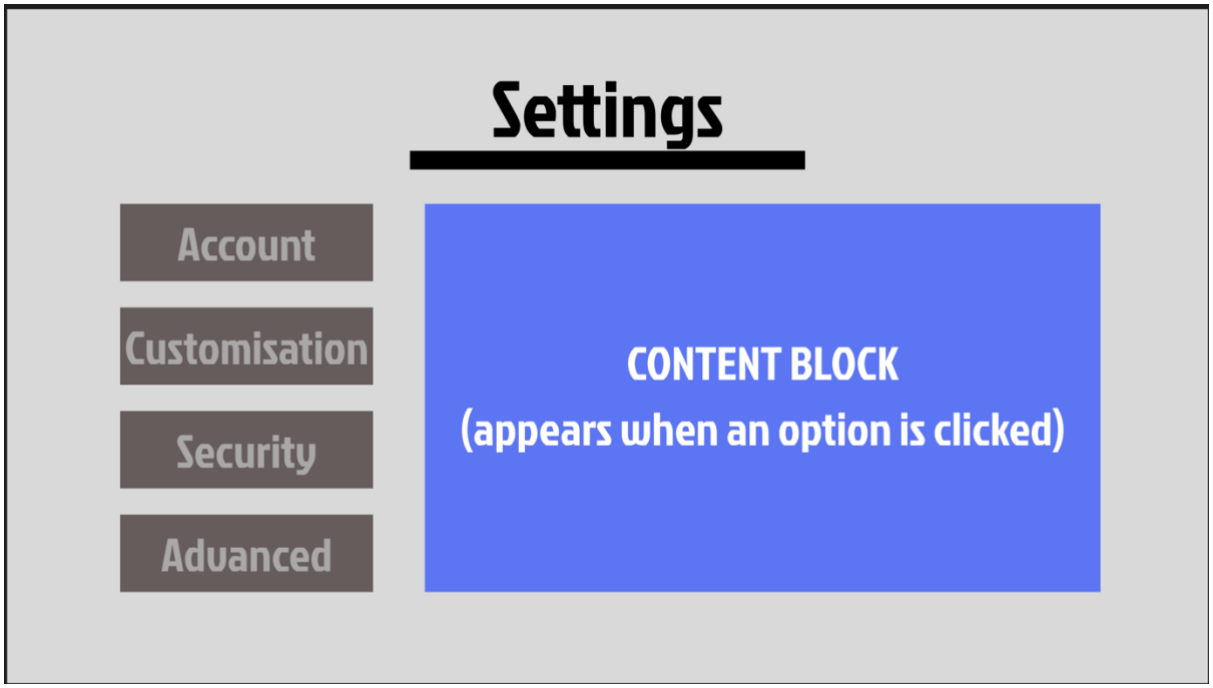

Figure 5 – Wireframe, Stronghold Settings Page

Figure 5 shows the wireframe for the settings page. This page was designed to allow the user to adjust a variety of settings for the browser, such as their theme, account details, and security settings. The initial design remained simple, relying on a changing content block to display the actual settings options based on the category selected by the user.

3.1.6 Blocking Page

Figure 6 – Wireframe, Stronghold Blocking Page

Figure 6 shows the wireframe for the blocking page. This page was designed to be simple and explain the reasons why a URL was blocked with an option to return to the previous page. This page was not intended to give the user the option to continue to the page was it was planned to be used only against incredibly high-risk pages.

3.1.7 Warning Page

Figure 7 – Wireframe, Stronghold Warning Page

Figure 7 shows the wireframe for the warning page. This page was designed to be simple and explain the reasons why a URL was warned against with an option to return to the previous page or continue to the page trying to be accessed. It was planned to be used only against medium risk pages that could be malicious but also contained many legitimate features.

3.2 Flowcharts

3.2.1 Risk Score Calculation

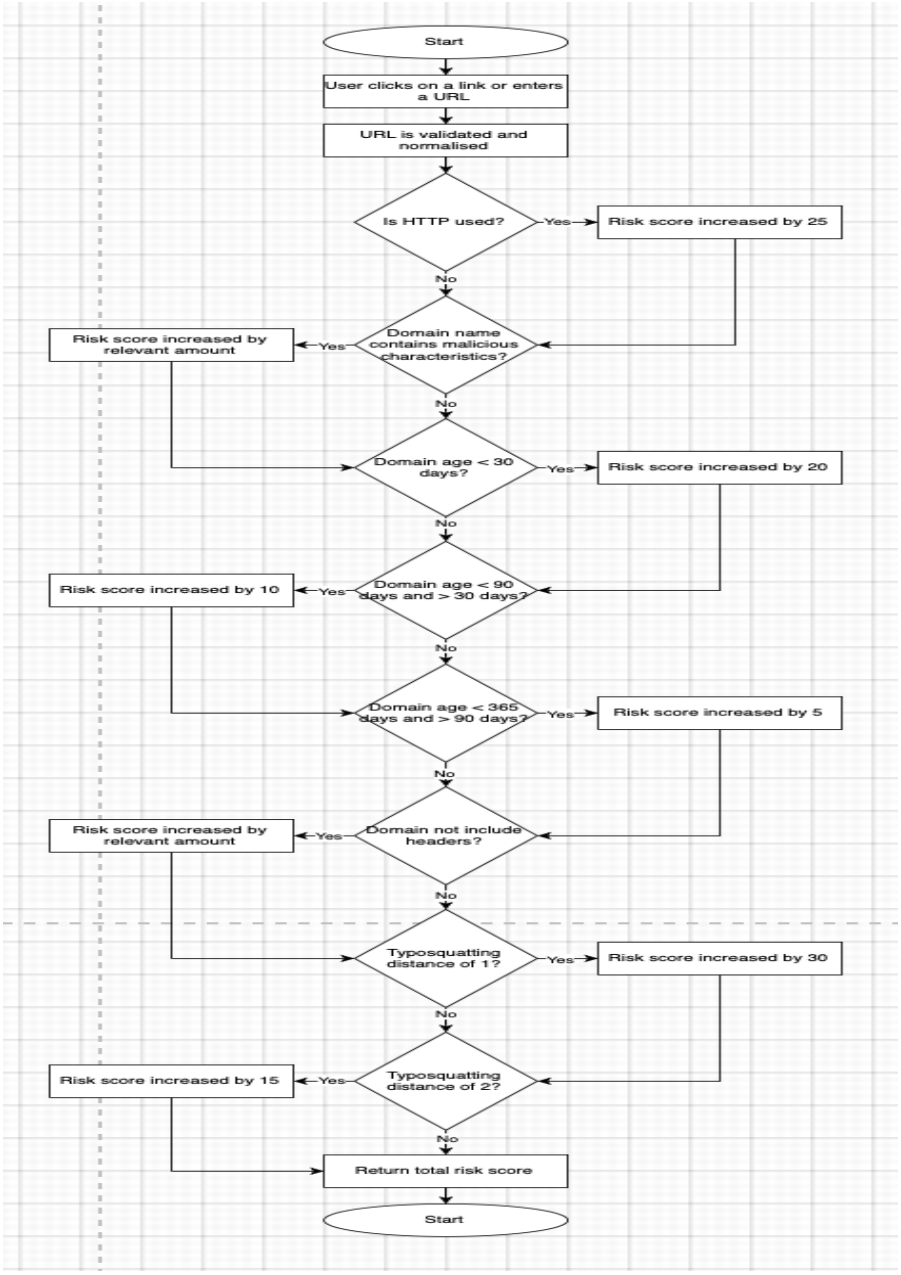

Figure 8 – Flowchart, Risk Score Calculation Process

Figure 8 shows the flowchart design for the risk score calculation process. This process begins once a user has clicked on or entered a URL into the search bar, and involves multiple security checks being performed, including use of HTTP, the structure of the domain, missing security headers, and similarity to a list of known domains. The final risk score is then used in navigation to determine whether the site should be loaded normally, warned against, or completely blocked.

3.2.2 Download Blocking Process

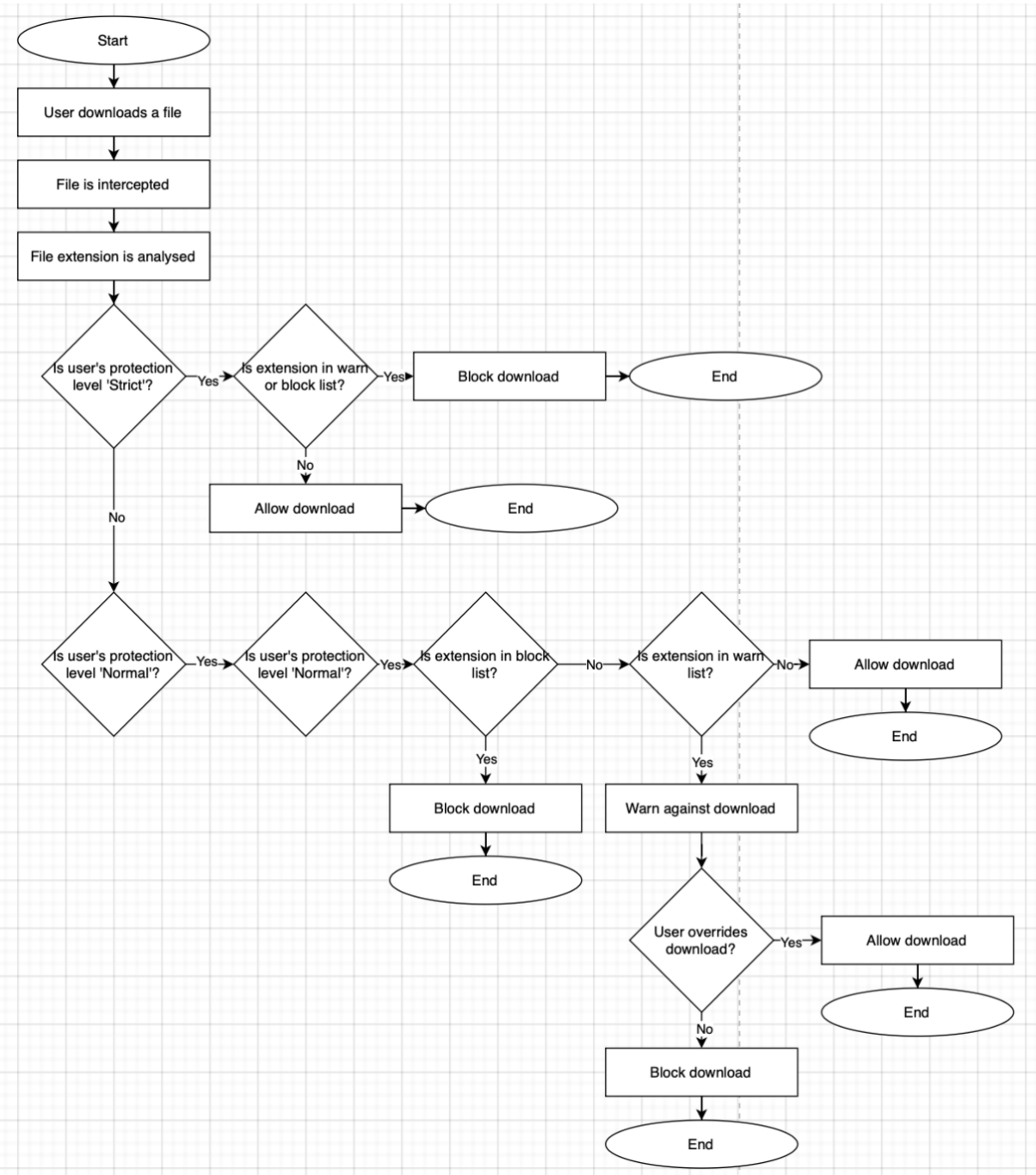

Figure 9 – Flowchart, Download Blocking Process

Figure 9 shows the flowchart design for the download blocking process. This process begins when a user attempts to download a file from a website. The file is intercepted before it can be downloaded and has its extension analysed against a predetermined list, which depends on the user's protection level. If deemed safe, the file is allowed to be downloaded; however, if deemed malicious, the file is either warned against or blocked based on the user's settings.

3.2.3 Navigation Flow

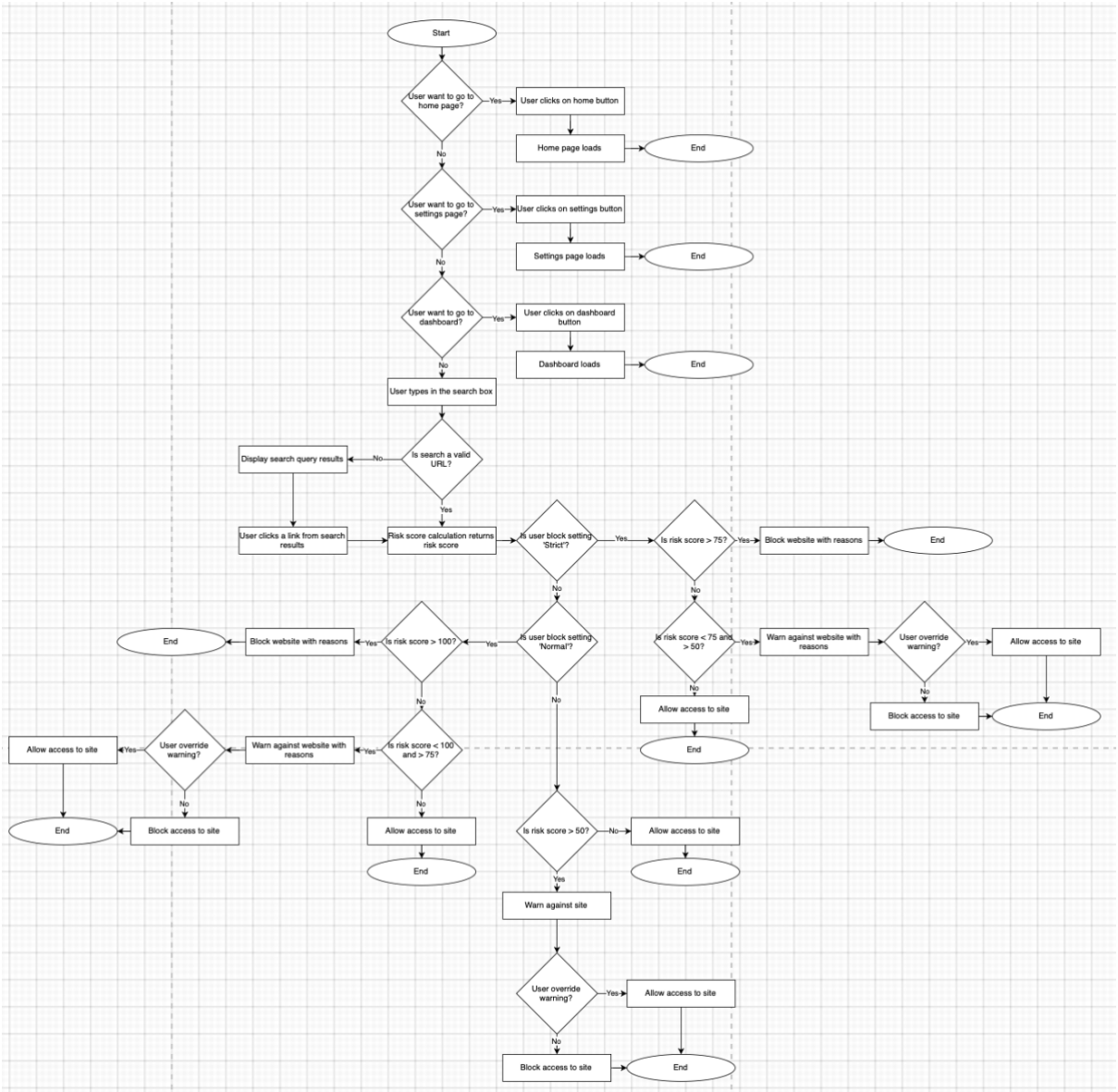

Figure 10 – Flowchart, Navigation Flow

Figure 10 shows the flowchart design for the navigation flow. This process begins when the user clicks on a button and details which pages get navigated to depending on the button clicked. This process can also begin when a user enters or clicks on a URL and compares the risk score of the site from the risk score calculation compared with the user's settings to allow access to the site, a warning page if deemed suspicious, or a blocking page if deemed almost certainly malicious.

3.2.4 Quiz Process

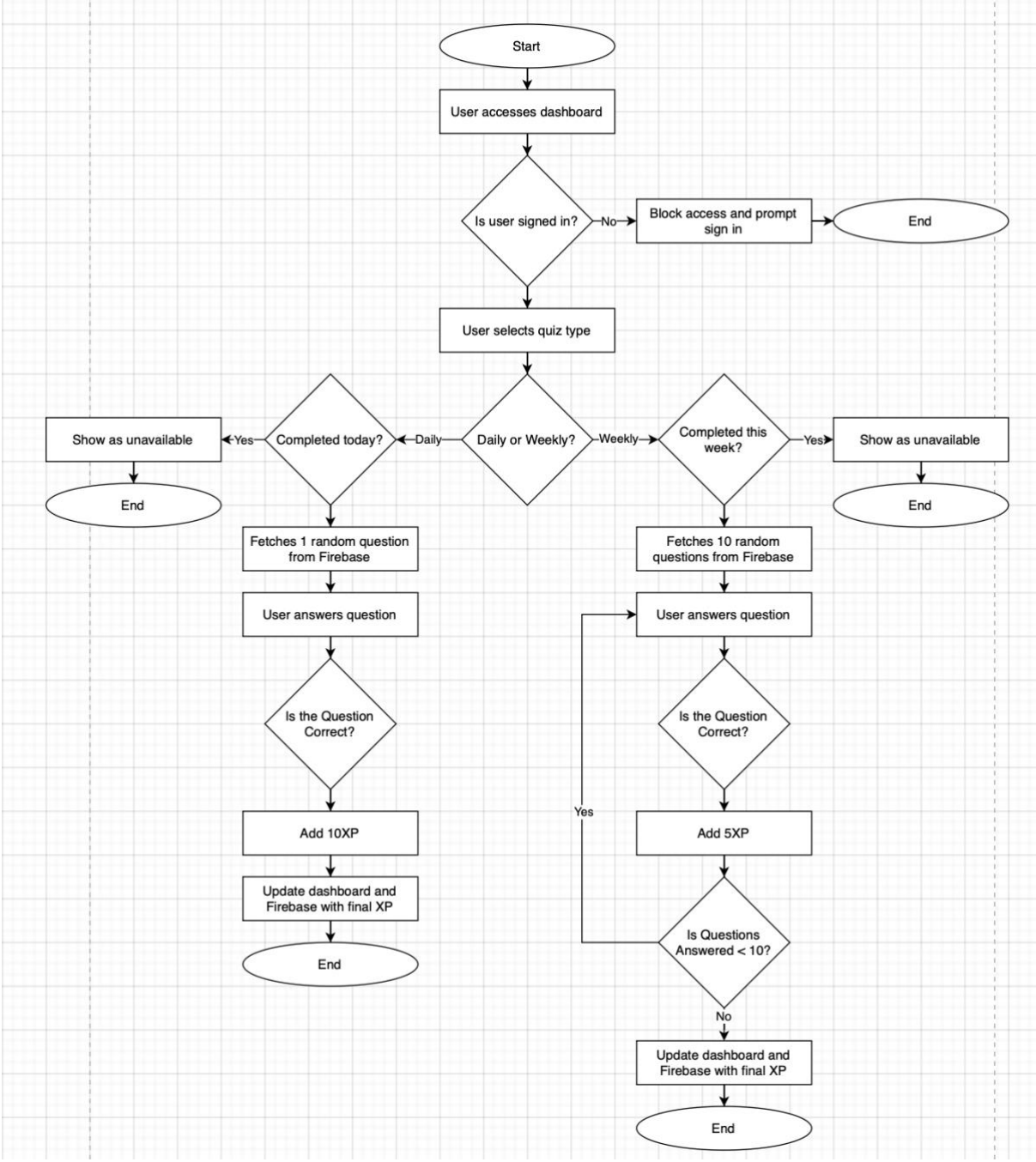

Figure 11 – Flowchart, Quiz Process

Figure 11 shows the flowchart for the quiz process. This process begins when the user accesses the dashboard, ensuring that they are first signed in. They are then prompted to select which quiz type they would like to complete, with daily quizzes being one question rewarding 10 experience points for a correct answer, and weekly quizzes being 10 questions and rewarding 5 experience points per correct answer.

3.3 Use Cases

3.3.1 Load Website

- The user enters a website URL in the search bar, or clicks on a URL from a search query
- The system analyses the URL format if it is manually typed by the user to ensure it is valid
- Before loading the page, the system checks the URL for a user-determined risk score
- If the URL is deemed as safe, it is loaded
- If the URL is deemed as potentially malicious, a warning page is loaded, explaining the reasons for the warn and allowing an override onto the website
- If the URL is deemed dangerous, a blocking page is loaded, explaining the reasons for the block with no override options

3.3.2 Download File

- The user clicks on a download link from a website
- The system intercepts the file before the download can begin
- The file is analysed based on its extension and the user's customised risk profile
- If the download is deemed safe, it is allowed and stored in the user's downloads folder
- If the download is deemed potentially risky, the user is shown a warning (if enabled) with the reason why and an override
- If the download is deemed dangerous, it is blocked (if enabled) and the user is shown the reason why with no override

3.3.3 Guest User Browsing

- The user wants to browse without signing in
- The user clicks on guest browsing button upon opening browser
- The user gains restricted access to the browser, with strict URL and download protection settings enabled
- Access to dashboard and settings menu are restricted behind a login

3.3.4 View Feedback

- The user accesses a site with a high-level risk score
- The website is warned or blocked, displaying the relevant page to the user
- The page displays the overall risk score along with the specific reasons for the warning or blocking of the site
- User gains a better understanding of why they should not be accessing that website to improve their browsing security in future

3.3.5 Typosquatting Detection

- The user attempts to access a malicious website with a URL close to a legitimate one
- The system compares the URL with a pre-defined list of similar domain names using the Damerau-Levenshtein algorithm
- The risk score is increased based on similarity, with the reason being added to the warning or blocking page if displayed

3.3.6 Phishing Detection

- The user attempts to access a malicious website with a URL that contains commonly used phishing words
- The system compares the URL to a pre-defined list of phishing words
- The risk score is increased if a phishing word is detected, with the reason being added to the warning or blocking page if displayed

3.3.7 TLS Certificate Validation

- The user attempts to access a website without a valid TLS certificate
- The system checks for a certificate and analyses its validity
- The system blocks the website and displays the blocking page with the reason

3.4 Unique Selling Points

Stronghold contains a variety of unique features which differentiates it from traditional web browsers like Chrome or Edge, focussing on education over background protection. Behavioural conditioning is used by encouraging positive

behaviour and punishing negative behaviour by manipulating experience points; this is something that other browsers do not provide.

Key features which provide a unique addition to Stronghold include the centralised dashboard which details the user's security actions, a reward-based levelling system where the user's experience level is increased or decreased based on their browsing actions, and interactive security-focussed quizzes designed to reinforce the information learned while browsing. These features aim to take a simple background feature for other browsers and turn it into a learning experience helping to keep younger people safer while online.

4 Method of Approach

This section will discuss the methodology used throughout the development of the project, explaining the architecture used to develop this project, including for the main code and the database, the decision making behind some of the core security features, and a discussion on future enhancements which could be made to the browser.

4.1 Risk Assessment

Before development could begin, a risk assessment was conducted to identify potential issues that may arise and impact the success or quality of this project. Table 1 shows the risks and evaluates them based on the likelihood of them occurring, the potential impact they may have on this project, and some mitigation techniques that were used to prevent the risks from causing an issue. Overall, these risks were mitigated or considered to be acceptable within the scope of this project, leading to a successful development cycle for relevant features.

Table 1 – Risk Assessment

Risk	Likelihood	Impact	Mitigation
Sickness	High	Low	Reduce factors that can cause sickness
Burnout	Medium	High	Pace development and take regular breaks

Time Constraints	High	Very High	Development of core features was prioritised by breaking down features into must have features and may have features
Feature Creep	Very High	Very High	Core features were laid out at the beginning of development and were heavily considered before adding new features to the project
Lack of Targeted Testing	Very High	Medium	Testing was performed with available users and assumptions were made about how this can suit a younger audience
Dependence on Libraries	High	Very High	Reliable libraries were selected and backups were implemented in the event of a misconnection
GDPR Issues	Very Low	Very High	Minimal data was stored and a privacy policy was provided, as well as the ability to delete an account
Cross Platform Issues	Low	Medium	Testing was performed on both Windows and MacOS devices
Firebase Integration Errors	Medium	Very High	Firebase connection was tested many times throughout development to ensure each additional feature worked with the integration

4.2 Agile Method

The agile method[20] was selected as the main development approach for this project due to the constant change in requirements, and the abundance of features needing to be developed, including general browser navigation, Firebase connection, a risk scoring system, a quizzing system, and a gamified dashboard. Using an iterative method like agile allowed for features to be developed, tested, and redesigned within allocated slots, to allow full focus into that feature.

Development was divided into sprints, starting with a focus on planning, research and design, and later moving into feature development and implementing core functionality. Trello[21] was used to track these sprints by planning tasks that should be completed within them, and marking off which tasks had been completed after the sprint was over.

Due to the lack of a development team, a full agile scrum method was not used; however, fortnightly meetings were arranged with a project supervisor to outline what had been completed, and what the next steps were for this project. This was useful to receive feedback on features and determine any changes which needed to be made, for example implementing a full risk score calculation over just blocking HTTP sites.

Overall, an agile method of development was important for receiving consistent feedback on implemented features and allowing for time to be focussed primarily on specific features to complete them to the highest level.

4.3 Tools and Technologies

4.3.1 JavaScript

JavaScript[22] was the predominant language used for the creation of technical features within this project, as well as the standard navigation. This was chosen due to its easy compatibility with Electron and its built-in commands.

4.3.2 Electron

Electron[23] was the main framework used for this project. Electron is a Chromium framework which, when used alongside JavaScript, allows for the creation of apps by utilising certain events and renderers to allow for navigation and other browsing features to be implemented. Electron was chosen due to its primary use being in desktop applications, as opposed to other frameworks such as Chromium Embedded Framework which is used primarily for embedding Chromium into existing applications.

4.3.3 Visual Studio Code

Visual Studio Code[24] was the IDE of choice for the creation of this project due to having prior experience with this IDE in past projects, and it already being configured on the devices used for this project.

4.3.4 Firebase

Firebase[25] was the platform of choice to host the database for this project due to its ability to automatically hash passwords, which is vital for ensuring security, and allowing for Google OAUTH authentication to be used for a single sign on into the browser. It was selected over options such as a locally hosted database due to the reduction of needing an additional account to be created specifically for Stronghold.

4.3.5 Devices

Both a Windows desktop PC and a MacOS laptop were used throughout development of this project. This allowed for cross-platform development, and assurance that this project would work on multiple devices. No Linux devices were used due to not being accessible at any point in development.

4.3.6 HTML and CSS

HTML[26] was used to create the browser-specific web pages, those being the home page, dashboard, settings menu, taskbar, and warning pages. CSS[27] was then used to design and style these HTML pages.

4.3.7 Figma

Figma was used to create the initial wireframe designs for this project which acted as a guide for when implementing the first draft of the HTML and CSS.

4.4 Risk Score Calculation

A risk score was calculated whenever a user attempted to access a website, which determined how secure the website was and therefore whether access should be granted, a warning should be provided, or access should be blocked. Figure 12 shows the code for the function ‘riskScore()’.

```
async function riskScore(input) {
    let score = 0;
    let action = 'allow';
    let reasons = [];

    const checkHTTPResult = checkHTTP(input);
    const checkDomainNameResult = checkDomainName(input);
    const checkDomainAgeResult = await checkDomainAge(input);
    const checkSecurityHeaderResult = await checkSecurityHeader(input);
    const typosquattingCheckResult = typosquattingCheck(input);

    score += checkHTTPResult.score;
    score += checkDomainNameResult.score;
    score += checkDomainAgeResult.score;
    score += checkSecurityHeaderResult.score;
    score += typosquattingCheckResult.score;

    reasons = [
        ...(checkHTTPResult.reasons),
        ...(checkDomainNameResult.reasons),
        ...(checkDomainAgeResult.reasons),
        ...(checkSecurityHeaderResult.reasons),
        ...(typosquattingCheckResult.reasons)
    ]

    const {warnScore, blockScore} = getProtectionLevel(userProtectionLevel);

    if(score >= blockScore) {
        action = 'block';
    }

    else if(score >= warnScore) {
        action = 'warn';
    }

    return {
        score,
        action,
        reasons
    };
}
```

Figure 12 – The riskScore() function integrated in Visual Studio Code

The risk score function was comprised of multiple checks which culminated in a total score that determined the safety of the site, applying an event based on the result before the site was loaded. The features used for the risk score were:

4.4.1 HTTP Check

Websites utilising HTTP over HTTPS are flagged as a higher risk due to the lack of secure encryption on the site[28], increasing the risk score by 25 points. Figure 13 shows the code for the function 'checkHTTP()'. While this is not necessarily indicative of a malicious site, the increase in risk score is in place since users should not be entering login details on these pages due to them being sent to the web server in plaintext, which can risk interception.

```
function checkHTTP(input) {
    let score = 0;
    let reasons = [];

    try {
        const formatURL = new URL(input.trim());

        if(formatURL.protocol === 'http:') {
            score += 20;
            reasons.push('Uses HTTP Protocol - Should use HTTPS instead')
        }

        return {
            score,
            reasons
        };
    }

    catch {
        return {
            score,
            reasons
        };
    }
}
```

Figure 13 – The checkHTTP() function integrated in Visual Studio Code

4.4.2 Domain Name Check

A domain name checking function was implemented to determine whether a URL contains characteristics frequently hinting to malicious behaviours. This involved

extracting the domain name from the URL and comparing it to multiple pre-determined risk factors. Figure 14 shows the code for the function 'checkDomainName()'. Table 2 shows which risk factors were considered to determine if a domain may be malicious[29].

Table 2 – Domain Name Check Risk Factors

Factor	Condition	Score Adjustment	Reason
Phishing Words	Contains phishing words i.e. 'login', 'secure', or 'verification'	+10 points	Commonly used to imitate legitimate services and deceive users
Repeated Hyphens	Contains multiple hyphens in a row	+5 points	Used to replicate legitimate domains
Long Domain Name	Domain name exceeds 50 characters	+10 points	Can blend in malicious pathways and avoid traditional security filters
IP Address	URL is an IP address	+30 points	Conceals domain name and may indicate a maliciously hosted server
Homograph Attack	Contains punycode 'xn--'	+30 points	Exploits similar looking characters to impersonate legitimate domains
@ Symbol	Contains '@'	+20 points	Obscures destination by placing it after a legitimate domain
Multiple Dots	Contains three or more dots	+5 points	Enables subdomain manipulation to imitate legitimate domains

```
function checkDomainName(input) {
    let score = 0;
    let reasons = [];

    try {
        const formatURL = new URL(input);
        const domainName = formatURL.hostname.toLowerCase(); // Changes the URL to be lower case
        const ipFormat = /^\d{1,3}(\.\d{1,3}){3}$/; // Sets the format of an IP address for comparison with URL
        const dotCount = (domainName.match(/\./g)).length; // Sets the format for multiple dots for comparison with URL


        // Updates risk score if the URL contains common phishing words
        if(phishingWords.some(word => domainName.includes(word))) {
            score += 10;
            reasons.push('URL contains common phishing words - This could lead to a scam or details being stolen');
        }

        // Updates risk score if the URL contains multiple hyphens
        if(domainName.includes('--')) {
            score += 5;
            reasons.push('URL contains multiple hyphens - This could be attempting impersonation of a legitimate URL');
        }

        // Updates risk score if the URL is longer than 50 characters
        if(domainName.length > 50) {
            score += 10;
            reasons.push('URL is longer than 50 characters');
        }

        // Updates risk score if the URL is an IP address
        if(ipFormat.test(domainName)) {
            score += 30;
            reasons.push('URL is an IP address - This could be a malicously hosted web domain');
        }

        // Updates risk score if there is a suspected homograph attack (browsers represent unicode as 'xn--')
        if(domainName.includes('xn--')) {
            score += 30;
            reasons.push('URL contains the broswer unicode representation - This could be an attempt at typosquatting');
        }

        // Updates risk score if URL contains an @ symbol
        if(input.includes('@')) {
            score += 20;
            reasons.push('URL contains the @ symbol - This could be an attempt at stealing data');
        }

        // Updates risk score if URL contains multiple dots
        if(dotCount > 3) {
            score += 5;
            reasons.push('URL contains multiple dots - This could be an attempt to hide additional redirects');
        }

        return {
            score,
            reasons
        }
    }

    catch {
        return {
            score,
            reasons
        };
    }
}
```

Figure 14 – The checkDomainName() function integrated in Visual Studio Code

4.4.3 Domain Age Check

The WhoIs API[30] is used to check the age of a domain, increasing the risk score based on the age. Figure 15 shows the code for the function 'checkDomainAge()'. The age risk is split into four categories: younger than 30 days (adding 20 points); between 30 and 90 days (adding 10 points); between 90 and 365 days (adding 5 points); and older than 365 days (adding no points).

```
async function checkDomainAge(input) {
    let score = 0;
    let reasons = [];

    try {
        const formatURL = new URL(input);
        const domainName = formatURL.hostname.toLowerCase();
        const whoIsResult = await whois(domainName);

        const createdDate = new Date(whoIsResult.creationDate); // Fetches the domain's creation date from WhoIs
        const todayDate = new Date(); // Fetches the current date
        const age = ((todayDate - createdDate) / (1000 * 60 * 60 * 24)); // Calculates the age of the domain in days

        // Updates as the highest risk if the domain is under 30 days old (1 month)
        if(age <= 30) {
            score += 20;
            reasons.push('Domain is younger than 1 month old');
        }

        // Updates as a medium risk if the domain is between 30 and 90 days old (1 month - 3 months)
        else if(age <= 90 && age > 30) {
            score += 10;
            reasons.push('Domain is older than 1 month but younger than 3 months');
        }

        // Updates as a low risk if the domain is between 90 and 365 days old (3 months - 1 year)
        else if(age <= 365 && age > 90) {
            score += 5;
            reasons.push('Domain is older than 3 months but younger than 1 year');
        }

        // Does not increase risk if the domain is older than 365 days (1 year)
        return {
            score,
            reasons
        };
    }

    catch {
        return {
            score,
            reasons
        };
    }
}
```

Figure 15 – The checkDomainAge() function integrated in Visual Studio Code

4.4.4 Security Header Check

Before a site is loaded, the HTTP headers are fetched, and the risk score is increased based on which security headers are missing, with different headers adding different weighting based on importance as some headers are used in

specific contexts. Figure 16 shows the code for the function 'checkSecurityHeader()', which compares against a list of all currently utilised security headers. Table 3 shows each of the headers involved in this check and their relevance for security of a website, along with drawbacks for those that have them[31].

Table 3 – Security Headers Checked

Header	Score Adjustment	Reason	Drawbacks
x-frame-options	+5 points	Used to avoid clickjacking attacks by ensuring content is not embedded into other sites	Only useful when there is something to interact with i.e., a link or a button
x-xss-protection	+2 points	Stops pages from loading if a cross-site scripting attack (XSS) is detected	Can create XSS vulnerabilities in safe websites
x-content-type-options	+5 points	Used to block MIME type sniffing and MIME confusion attacks	N/A
referrer-policy	+2 points	Controls how much referrer information should be included with requests	Default behaviour is to no longer send all referrer information to the same site
strict-transport-security	+10 points	Instructs access to be only via HTTPS even if HTTP is attempted by the user	Misconfiguration may lead to a legitimate user being blocked from the website
content-security-policy	+10 points	Detects and mitigates attacks including XSS and data injection attacks	Meaningless if REST API returns content that will not be rendered
cross-origin-opener-policy	+2 points	Protects against attacks like Spectre	N/A
cross-origin-embedder-policy	+2 points	Prevents a document from loading cross-origin resources that don't grant the document permission	Enabling will block cross-origin resources not configured correctly from loading

cross-origin-resource-policy	+2 points	Blocks a given response before entering an attacker's process	N/A
permissions-policy / feature-policy	+2 points	Allows features to only be enabled in the current document or frame if it matches the list of allowed origins	N/A

```
async function checkSecurityHeader(input) {
    let score = 0;
    let reasons = [];

    try {
        const fetchedResponse = await fetch(input, {method: 'GET'}); // Fetches the HTTP headers from the domain
        const responseHeaders = fetchedResponse.headers;

        // Checks for the x-frame-options header
        if(!responseHeaders.has('x-frame-options')) {
            score += 5;
            reasons.push('Domain does not have the x-frame-options security header');
        }

        // Checks for the x-xss-protection header
        if(!responseHeaders.has('x-xss-protection')) {
            score += 2;
            reasons.push('Domain does not have the x-xss-protection security header');
        }

        // Checks for the x-content-type-options header
        if(!responseHeaders.has('x-content-type-options')) {
            score += 5;
            reasons.push('Domain does not have the x-content-type-options security header');
        }

        // Checks for the referrer-policy header
        if(!responseHeaders.has('referrer-policy')) {
            score += 2;
            reasons.push('Domain does not have the referrer-policy security header');
        }

        // Checks for the strict-transport-security header
        if(!responseHeaders.has('strict-transport-security')) {
            score += 10;
            reasons.push('Domain does not have the strict-transport-security security header');
        }

        // Checks for the content-security-policy header
        if(!responseHeaders.has('content-security-policy')) {
            score += 10;
            reasons.push('Domain does not have the content-security-policy security header');
        }

        // Checks for the cross-origin-opener-policy header
        if(!responseHeaders.has('cross-origin-opener-policy')) {
            score += 2;
            reasons.push('Domain does not have the cross-origin-opener-policy security header');
        }

        // Checks for the cross-origin-embedder-policy header
        if(!responseHeaders.has('cross-origin-embedder-policy')) {
            score += 2;
            reasons.push('Domain does not have the cross-origin-embedder-policy security header');
        }

        // Checks for the cross-origin-resource-policy header
        if(!responseHeaders.has('cross-origin-resource-policy')) {
            score += 2;
            reasons.push('Domain does not have the cross-origin-resource-policy security header');
        }

        // Checks for the permissions-policy header
        if(!responseHeaders.has('permissions-policy') && !responseHeaders.has('feature-policy')) {
            score += 2;
            reasons.push('Domain does not have the permissions-policy security header');
        }

        return {
            score,
            reasons
        };
    }

    catch {
        return {
            score,
            reasons
        };
    }
}
```

Figure 16 – The checkSecurityHeader() function integrated in Visual Studio Code

4.4.5 Typosquatting Check

Typosquatting involves masking a malicious URL to look like a legitimate one, by exploiting misspellings of common domains. The Talisman package[32] was used to create a typosquatting check, utilising the built-in Damerau-Levenshtein function to determine the distance between the entered URL and a list of common URLs. The risk score is then increased depending on how close the domains are, increasing by 30 points if the domain is one step away, and by 15 points if the domain is two steps away. This utilises only the closest distance to prevent the score being increased multiple times from checking multiple domains, The domain was split to extract subdomains, which allowed checks to be performed on each subdomain to analyse its distance. If the URL matches exactly, the function is not called to prevent any false positives from being created and access to a legitimate domain being blocked. Figure 17 shows the code for the function ‘typosquattingCheck()’.

The Talisman package was used to import the Damerau-Levenshtein algorithm, which was chosen over other packages due to it containing the largest library for comparison. Additionally, when compared to other algorithms such as fast-levenshtein[33] or js-levenshtein[34], Damerau-Levenshtein is the most optimal for detecting swapped letters between domain names.

```
function typosquattingCheck(input) {
    let score = 0;
    let reasons = [];

    try {
        const formatURL = new URL(input);
        const domainName = formatURL.hostname.toLowerCase().replace(/^www\./, '');
        const domainSplit = domainName.split('.');
        const host = parse(domainName).domainWithoutSuffix;

        if(typeof host === 'string' && host.length > 0) {
            if(knownDomains.includes(host)) {
                return {
                    score,
                    reasons
                };
            }

            // Compares how far away from a known domain the user entered domain is
            for(const trustedDomain of knownDomains) {
                const distance = damerauLevenshtein(host, trustedDomain);

                // Flags the domain as high riks if it is only one score away from known domains
                if(distance === 1) {
                    score = Math.max(score, 30);
                    reasons.push('Domain name is very close to another - Likely impersonation');
                }

                // Flags the domain as medium riks if it is two scores away from known domains
                else if(distance === 2) {
                    score = Math.max(score, 15);
                    reasons.push('Domain name is failry close to another - Could be impersonation');
                }
            }
        }

        // Compares how far away from a known domain the user entered subdomain is
        for(const subDomain of domainSplit) {
            if(subDomain === host) {
                continue;
            }

            if(typeof subDomain !== 'string') {
                continue;
            }

            for(const trustedDomain of knownDomains) {
                const distance = damerauLevenshtein(subDomain, trustedDomain);

                if(distance === 1) {
                    score = Math.max(score, 30);
                    reasons.push('Subdomain is very close to another - Likely impersonation');
                }

                else if(distance === 2) {
                    score = Math.max(score, 15);
                    reasons.push('Subdomain is fairly close to another - Could be impersonation');
                }
            }
        }

        return {
            score,
            reasons
        };
    }

    catch {
        return {
            score,
            reasons
        };
    }
}
```

Figure 17 – The typosquattingCheck() function integrated in Visual Studio Code

5 Implementation

5.1 Features

This section details the features that were implemented into the final project. The features are focussed primarily on security due to the nature of this project, with explanations as to why they are necessary for the final implementation.

5.1.1 Session Only Cookies

To allow for full privacy to be kept, Stronghold automatically deletes cookies once the browser is closed and the session has ended. This means that cookies are not stored, which can assist in mitigating the damage from an attack which relies on stealing cookies, as only cookies from the current session will be available – This could include a cookie scraper[35], which would take a user's cookies, including login tokens that would enable unauthorised access to that user's sessions for malicious purposes.

This feature has been fully implemented in the final product. Initially, this feature was intended to be customisable from within the settings menu, allowing the user to decide whether it should be enabled or not. This was changed in the final implementation to ensure guaranteed deletion for security purposes. This could however lead to user irritation, due to not being able to save logins, and having to accept website cookies on every visit.

5.1.2 Cookie Export Authentication

Exportation of cookies was a potential vulnerability identified against this project, as this could lead to session tokens being stolen[36] which can then allow for malicious access into user's accounts. While session only cookies were implemented as an attempt to stop this, having authentication being needed to export these cookies could act as an extra layer to mitigate the opportunity for these attacks to succeed.

The initial design for this feature was to work alongside the toggle for enabling cookies. However, since this was never implemented and was instead changed to always delete cookies at the end of a session, authentication for exporting was never implemented. This was initially planned as a stretch goal and was deemed as unnecessary based on how cookie handling was implemented.

5.1.3 Blocking of Non-HTTPS Sites

Due to the nature of HTTP sites not containing relevant security features, this project initially involved a block on all HTTP sites to allow for an improvement in security. This aimed at reducing the risk of users accessing websites that could lead to plaintext interception of confidential details such as login details, although this resulted in blocking many legitimate sites where high security is not deemed necessary due to the nature of the site.

While not implemented as initially planned, HTTP blocking was involved as part of the risk blocking system that had been developed for this project. The final implementation allowed for HTTP-based sites to still be accessed by the user while increasing the risk score. This meant the site would only become blocked assuming other aspects of the risk scoring system were increased.

5.1.4 Blocking Downloads

A security feature deemed necessary for this project is to block downloads of file extensions which could potentially be used for malicious purposes*[37]*, and warning the user with an informative message to explain why. This includes extensions such as .exe, .msi, .cmd, and .zip. By blocking these types of extensions, this will prevent any accidental downloading of malicious files, especially with the target audience being children who do not fully understand the dangers of the internet.

This feature was implemented in the final design; however, it did receive some changes compared to the initial plan. Due to the how all extensions were treated the same and were automatically blocked, this led to a lot of legitimate files being blocked also, such as Microsoft Office*[38]* downloads which utilise the .exe file extension. A change was made in the final implementation to accommodate this by allowing a selective toggle in the settings menu for multiple tiers of download blocking.

A user can select between the following levels: strict blocking, wherein all downloads with commonly malicious extensions are blocked; normal blocking, wherein certain extensions, such as .exe, .msi, .dmg, and .zip, are warned against, while others, such as .cmd, .ps1, .bat, and .js, are blocked; and warn only blocking, wherein all downloads with commonly malicious extensions are only warned against.

5.1.5 TLS Certificate Checking

TLS certificates[39] are mandatory for HTTPS sites to prove they are trustworthy to access. As such, when a site has a poorly configured TLS certificate, this should be marked as high-risk. This project includes a check to ensure that a site has a correctly configured TLS certificate when the user attempts to access it and blocks the site if the certificate is not correctly configured. A misconfigured TLS certificate includes being expired, being for a different domain, and not being issued from an approved issuer. Assuming a site is supposed to have a certificate, this check is performed and either blocks or allows access to the site based on the status of the certificate, allowing the user to remain safe and clear of potentially malicious domains.

5.1.6 Security Based Hints

An initial plan was to have hints pop up as the user navigated the app, providing knowledge about certain security elements to teach the user about online safety. This would fit in with the initial objectives outlined at the beginning of this report. Instead, the feature has been implemented in a way where it is explained to the user why a site has been warned or blocked, but there are no pop-ups that appear generally as they browse.

5.1.7 Override Methods

Override methods were an essential feature to implement into this project, as this allows users to continue to a page or download that is deemed potentially malicious and therefore receives a warning by Stronghold.

This feature initially had been planned to require a parental control PIN or one time code to be entered to override the warning. Since these parental controls were never implemented, this was unable to happen so the final implementation includes a simple page with a warning and explanation as to why the block has taken place. The user can acknowledge this warning to continue, causing a small amount of experience loss, or the user can go back, causing a small experience gain.

5.1.8 Quiz System

One of the unique selling points of this project involved having quizzes that the user could complete for bonus experience, to reinforce their learning. The quiz system implemented involved both daily and weekly quizzes.

The daily quizzes could only be completed once per day and were only one question, rewarding 10 experience points to the user, and was chosen from a random list of set questions. The final implementation included 5 questions stored in Firebase; however, this would be expanded in the future.

The weekly quizzes could only be completed once per week and were 10 questions long, rewarding 5 experience points per correct question to the user. These were also chosen at random from a set list of questions. The final implementation included 10 questions stored in Firebase; however, this would be expanded in the future. This has meant that the questions are all the same but in a random order for this final implementation.

5.1.9 Guest Browsing

Guest browsing was implemented as an additional security measure, to allow users access who do not wish to input an email address or sign in to their Google account. This feature involved blocking access to the settings menu and dashboard, requiring a sign in to be able to access these, since guests were restricted to the most basic, strictest forms of download and URL blocking.

5.1.10 Security Dashboard

A centralised security dashboard was implemented to inform users of their browsing habits and contain elements relating to the gamification of the browser. This dashboard acted as a unique selling point for the browser, providing both learning elements and security statistics. Users were able to view their five most recent blocked sites and blocked downloads, as well as recent ignored warnings. The dashboard also showed gamification elements which included one daily question and ten weekly questions, experience amounts, and recent experience changes that the user received.

5.2 Firebase

Firebase was used as the primary database to store user and quiz data. Appendix A shows screenshots from Firebase, including an example user and example quiz questions. Its use also enabled the addition of Google OAUTH[40] for logins, removing the need for a separate Stronghold account to be made. This was beneficial for this project as it removed the need to create additional hashing[41] and salting[42] algorithms for password security and decreased the amount of sensitive data needing to be stored and retrieved.

When signing in for the first time, a user document is made providing the user with a unique user ID that then enabled their specific details to be written and read. The document was created with some default variables required for security tracking and gamification: experience points, safe day streak, sites blocked, downloads blocked, warnings ignored, completed quizzes, and recent changes as examples. Default variables for settings were also included: download level, protection level, start page, and theme as examples. Settings data was chosen to store in Firebase instead of locally due to having multiple account sign-ins be available from the same device. These could then be manipulated upon user actions and could be retained between browsing sessions with their account login.

Quiz data used for daily and weekly quizzes were also stored within Firebase, including questions, answers, and experience rewarded upon correctly answering. Completion data was tracked within the user document to prevent repeated completion of the quizzes in the same day or week respectively.

Using Firebase allowed for the project to become more scalable in future due to separating user data from local browser files which meant that user data could exist across devices with the same progress in gamified elements. It also allowed for simpler updating of data since actions could be updated within Firebase that then pushed changes directly to the browser and forced updates live.

However, Firebase did introduce some limitations alongside its integration. The most notable of these was that users required a Google account to sign in and therefore access features, which may not be suitable for all users as not everyone has a Google account. Additionally, as Firebase is a cloud storage method, internet access

is required to read and write data. This is not a major issue however since an internet access would be needed for web browsing which is the main purpose of this project.

Overall, Firebase was deemed as the most suitable option for this project due to the authentication provided and persistent user data without needing a customised backend. This allowed development time to be focussed primarily on important gamified and security features.

5.3 Licenses

Throughout the project, several libraries and frameworks have been used which needed to be considered for licensing purposes. Electron[43], Talisman[44], WhoIs[45], and Firebase[46] all contain free use licensing which makes them suitable for development in this project.

6 LSEP Considerations

This section will discuss the legal, social, ethical, and professional issues which needed to be considered throughout the creation of this project to ensure this project did not break any laws, remained ethical throughout its creation, and could remain suitable for everyone and conform to societal and professional industry norms.

6.1 Legal Issues

Avoiding breaches of GDPR[47] was an issue that needed to be addressed throughout the creation of this project, as doing so could have resulted in significant financial penalties. This was ensured by collecting minimal data which was stored inside Firebase and not shared outside of the scope; only the user's username and data about Stronghold was collected. Furthermore, a privacy policy was implemented to outline to the user what data would be collected, how it would be stored, and what would happen to it. The user was also provided with an option to delete their account, and therefore their data, from within the settings menu.

Additionally, licensing issues needed to be addressed. With a variety of libraries being used to develop this project, it was essential to ensure they had valid licenses that would allow free use. This guaranteed there would not be any issues with the

necessary libraries for this project to work. Any breach of licensing terms could lead to lawsuits from the developer of the library; therefore, it was imperative that only free use libraries were used.

6.2 Social Issues

Accessibility needs[48] were considered throughout development of this project. These features would need to allow for a variety of accessibility needs to be met, including vision-based disabilities and learning disabilities. Using simplified language was suitable to assist people with learning disabilities by helping them to learn without being too imposing and creating frustration with the app. Contrasting colours were considered to assist users with vision-based disabilities which needed to be balanced with colours emitting a secure feeling. The final design was deemed as suitable for this, with an option considered for high contrast; however, this was not implemented in the final project.

Furthermore, it was essential to keep the browser age appropriate since it was developed with children as target audience. This was accomplished by using child-friendly language across educational segments which included simplifying more complicated theory to assist with learning and not using fear-based language which may cause more damage to online education than warranted.

6.3 Ethical Issues

Testing this project created some ethical issues, particularly when considering testing against the target audience. Due to the target age group involving children, there was no ethical way to accurately test against them, therefore it was slightly inaccurate due to being performed away from the target audience. Parents were considered to determine if they would allow their children to use the browser, however this required ethical consent from the university to perform.

Gamification caused some ethical concerns as it needed to be implemented without encouraging unhealthy behaviour such as negative competition or addiction. This was accomplished by keeping all gamified aspects simple and with limited repetition, such as having the quizzes be non-repeating within a specified length of time, either daily or weekly depending on the quiz. Additionally, the lack of cosmetic rewards adds to lowering the competitive elements, although this would be a feature

considered for future implementation and therefore would require further ethical analysis.

6.4 Professional Issues

Professional considerations focussed primarily on formal coding practices and clean documentation. These included using clear comments to accurately explain the code, using version control throughout development, and keeping documentation clear and concise. This has been accomplished by commenting functions carefully to explain the tasks they perform, using GitHub Desktop[49] to create a version history of development by pushing changes consistently, and documenting the creation process without repetition, spelling or grammar mistakes, and in an organised fashion.

7 Project Management

7.1 Sprints

This section outlines details of the sprints that took place throughout development of this project. Each section will outline the aims of the sprint, and whether those aims were achieved or not. Screenshots of the Trello board for each sprint can be found in Appendix B. Sprints started with an initial focus on design and planning stages as the objectives for the project had begun to be outlined. This then transitioned into the start of development and time assigned to write this report, with features being arranged in an organised manner to determine the importance and order to get this project working.

7.2 Trello

Trello was the project management tool used to plan each sprint at the beginning of development and explain what was performed once each sprint had finished. While the initial plan was not stuck to for each sprint, Trello was still useful at tracking which features had been worked on or implemented in these sprints.

7.3 GitHub Desktop

GitHub desktop was the software used throughout development to create a history of the different versions of this project. This was useful when changes needed to be

reverted due to bugs or changes suggested after user testing and was also beneficial to understand what was performed throughout each sprint.

Using a standardised naming scheme was helpful to establish professionalism throughout, although this was not always necessarily followed, as was descriptions of what changes were being made with each commit.

8 Testing and Evaluation

A combination of functional and user testing was performed to fully review initial wireframe designs, technical functionality, and the overall usability of Stronghold. This section will explain the methodology used to perform this testing.

The initial testing performed involved showing the wireframe designs created to a small group of 5 people to gain an understanding of what users liked and did not like about the user interface. This testing was performed by showing the small group the created wireframes and receiving verbal feedback, which was not documented. The conclusion from this initial testing was that the wireframes made the app look too dull, with changes suggested including adding more colour, rounding corners on buttons, and giving the app a more security-focussed feel instead of simple colours like black on grey. Appendix C shows user interface screenshots from Stronghold to reveal the changes that had been made because of this feedback. These changes included changing the background to navy blue, rounding the corners on buttons, and adding gradients and drop shadows to elements to make them more noticeable.

Functional testing was performed throughout development to confirm that implemented features performed their intended functions. This testing included navigation, URL warning and blocking, download warning and blocking, and quiz functionality among others. Table 4 shows the results from this functional testing. Each feature was tested using both expected and edge case scenarios, with intended outcomes being defined before testing and the real outcome being documented after testing. The result was then recorded to determine whether the intended and real outcomes matched, in which case the test passed, or did not match, in which case the test failed. The conclusion of these results show that all features implemented have performed as expected, which indicates that the main functional objectives were satisfied.

Table 4 – Functional Testing Requirements

Test Performed	Expected Outcome	Real Outcome	Result
Safe URL entered	Site loads	Site loaded	Pass
Risky URL entered	Warning URL page is displayed	Warning URL page was displayed	Pass
Malicious URL entered	Blocked URL page is displayed	Blocked URL page was displayed	Pass
Safe download attempted	Warning download page is displayed	Warning download page was displayed	Pass
Malicious download attempted	Blocked download page is displayed	Blocked download page was displayed	Pass
Browse as a guest	Strict browsing mode is enabled and access to dashboard and settings page is blocked	Strict browsing mode was enabled and access to dashboard and settings page was blocked	Pass
Sign in	Google OAUTH popup appears	Google OAUTH popup appeared	Pass
Delete account	Account is deleted from Firebase	Startup page was displayed and account was deleted from Firebase	Pass
Complete daily quiz once	Relevant XP is awarded and further access is blocked	Relevant XP was awarded and further access was blocked	Pass

Complete daily quiz twice	Cannot access	Could not access as button was disabled	Pass
Complete weekly quiz once	Relevant XP is awarded and further access is blocked	Relevant XP was awarded and further access was blocked	Pass
Complete weekly quiz twice	Cannot access	Could not access as button was disabled	Pass
Click forwards button	Navigates forwards	Navigated forwards	Pass
Click backwards button	Navigates backwards	Navigated backwards	Pass
Click refresh button	Refreshes page	Refreshed page	Pass
Click home button	Navigates to the home page	Navigated to the home page	Pass
Click dashboard button	Navigates to the dashboard	Navigated to the dashboard	Pass
Click settings button	Navigates to the settings page	Navigated to the settings page	Pass

User acceptance testing was then performed after development had been completed to ensure that users could perform a set of tasks modelled after the initial objectives set at the beginning of this report. 7 participants were provided with a Google form*[50]* containing multiple tests that they were tasked with completing to identify that the features of the browser were intuitive and simple to understand. Appendix D shows screenshots of the Google form provided which includes the tasks the participants were asked to perform. Overall, participants were able to perform the tasks to a suitable standard, with the results showing that the features of this project have been implemented to a suitable standard. Slight deviations existed in some of the questions; however, these instances involved questioning whether a participant

could access a page that had been warned against and so an assumption has been made that the user navigated backwards instead of continuing to the page. A final feedback box was provided at the end of the form for qualitative feedback to be provided, which primarily highlighted user enjoyment surrounding the changes made to the user interface design, although small changes were suggested to minor elements such as the delete account button.

Participants for this testing involved other university students, who were instructed to use Stronghold on the development device. While this limits the accuracy of testing, since younger users may not be as informed about how to use the browser, it still provides useful insight into user behaviour and mitigates the need for gaining ethical approval to test against children.

9 Future Implementations

9.1.1 DNS Check

The initial plan for the risk score involved performing a DNS check[51] on URLs that a user had attempted to access. This was planned as utilising a free online DNS checking tool to determine the safety of the domain by analysing if it is maliciously hosted, has a constantly changing IP address, or has missing DNS records. It was decided to not implement this feature as part of the risk score due to the integration required with an external source, and potentially reanalysing the domain for elements that the risk score already checks for, potentially falsely blocking domains. This remains as a feature for future implementation if an external source can be effectively utilised without causing repeat increases to the risk score.

9.1.2 Redirect Analysis

The initial plan for the risk score involved checking the number of redirects used in a URL to prevent a legitimate URL from redirecting to a malicious URL. Each redirect would be intercepted, with the risk score increasing exponentially per redirect used. This function was not implemented due to time constraints and the additional complexity that came with tracking redirects throughout the complete chain without incorrectly penalising legitimate websites using redirects for authenticity.

9.1.3 Parental Controls

Parental controls were considered to add an extra level of security for children who may be using this project. The initial design for parental controls was to have the user enter their date of birth upon account creation and requiring another sign in if detected as under 13 years old. After some consideration, it was decided that this needed to be changed since children could simply lie about their age or sign in with a separate Google account acting as the parent, both of which would negate this feature. Requiring an additional sign in would then have caused complications within Firebase due to having to link multiple Google sign in states together.

The workaround to this issue was to add a button in the settings menu, or prompted on first login, to add a PIN which would then be requested upon an attempt to change any security-based setting, such as changing the selected risk score algorithm or download blocking level. This had the same consideration however that the child may potentially set the PIN up themselves, or figure out what the PIN is, making the feature negligible.

This feature was not implemented in the final design due to the decision that it would not have the desired effect, and complications with linking the PIN to Firebase. This has meant that in the final implementation, there is no filter to prevent the settings from being changed undesirably as initially planned, potentially causing a loss in security.

9.1.4 AI Chatbot

An AI chatbot was considered for implementation as a stretch feature, which would act as a cyber security advisor. The plan for this feature involved allowing the AI to scan the page to confirm if URLs could potentially be malicious, both from web searches and in emails, as well as being able to answer user's questions. This AI could also be integrated to provide specific user feedback based on their browsing habits or tailoring the questions to a specific user based on how securely they browse. The Gemini API[52] was considered as a solution for implementing this feature, however it was not implemented due to time and difficulty constraints.

9.1.5 Built-in VPN

A built-in VPN was considered as an additional stretch feature for implementation. This would have involved integrating a free VPN provider to allow for use from within Stronghold, allowing for the additional security which VPNs provide. This was not implemented in the final project due to the high level of complexity and time required, and it being outside of the scope of the project.

10 Conclusion and Reflection

This project initially set out to design and develop a security-focussed web browser using the Electron framework which could teach online safety behaviours to a younger audience, utilising behavioural conditioning to do so. The final implementation of this project successfully completed this task by combining security features such as website blocking based on a risk score, download blocking based on extension, and TLS validation with gamification features such as quizzes and a levelling system.

A key strength was the combination of a traditional security browser with educational material to encourage the learning of safe browsing behaviour within children. Unlike other browsers which simply block based on security measures, Stronghold provides feedback to encourage learning, which fits the outlined objectives of promoting long-term changes in behaviour.

However, there were multiple limitations to Stronghold, which included the time constraints and complexity constraints forcing certain planned features to not be developed, such as a parental control PIN and an AI assistant to provide customised feedback.

The elements of this project that went well include being able to integrate all must have features into the final implementation within the allocated time, which shows strength in time management as the project has resulted in a working prototype that demonstrates the intended functionality.

If this project were to be worked on in future, these undeveloped features would be able to be implemented as there would be adequate time to research and integrate them in a suitable manner. While time management was strong enough to allow

must have goals to be integrated, if this project were to be reattempted, consideration could be given to stretch goals to allow them to be integrated by allocating time better throughout sprints. Furthermore, a small dataset was used for the quiz questions. If provided with more time, or if time was better allocated, this could be completed with a much larger dataset to reduce the chance of question repetition and therefore the chance for a user to stop completing the quizzes. An AI assistant could be linked to this also to make the questions more specific and advanced to the user.

Overall, Stronghold demonstrates the possibility to combine security mechanisms with learning and gamification aspects, while still providing a functioning web browser.

11 References

[1] Shang et al. (2022). *The psychology of the internet fraud victimisation of older adults: A systematic review* - Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC9484557/ (Last Accessed: 05-05-2026)

[2] The Old Station Nursery (2023). *Internet Safety | The Importance of Internet Safety for Children* – Available at: https://www.theoldstationnursery.co.uk/journal/internet-safety-the-importance-of-internet-safety-for-children/ (Last Accessed: 05-05-2026)

[3] Skinner (2003). *B. F. Skinner's Science and Human Behaviour: Its antecedents and consequences* – Available at: https://www.researchgate.net/publication/8692402_B_F_Skinner's_Science_and_Human_Behavior_Its_antecedents_and_its_consequences (Last Accessed: 05-05-2026)

[4] Holloway et al. (2013). *Zero to Eight: Young children and their internet use* – Available at: https://researchonline.lse.ac.uk/id/eprint/52630/1/Zero_to_eight.pdf (Last Accessed: 05-05-2026)

[5] IBM (2026). *X-Force Threat Intelligence Index 2026* – Available at: https://www.ibm.com/forms/mkt-1f268 (Last Accessed: 05-05-2026)

[6] Ouchbolagh et al. (2023). *Social influence in adolescence: behavioural and neural responses to peer and expert opinion* – Available at: https://www.researchgate.net/publication/372195439_Social_influence_in_adolescence_behavioral_and_neural_responses_to_peer_and_expert_opinion (Last Accessed: 05-05-2026)

[7] Verizon (2024). *2024 Data Breach Investigations Report* – Available at: https://www.verizon.com/business/resources/T6ac/reports/2024-dbir-data-breach-investigations-report.pdf (Last Accessed: 05-05-2026)

[8] National Cyber Security Centre (n.d.). *Device security guidance* – Available at: https://www.ncsc.gov.uk/collection/device-security-guidance/policies-and-settings/managing-web-browser-security (Last Accessed: 05-05-2026)

[9] Zscaler (2024). *Phishing Via Typosquatting and Brand Impersonation: Trends and Tactics* – Available at: https://www.zscaler.com/blogs/security-research/phishing-typosquatting-and-brand-impersonation-trends-and-tactics (Last Accessed: 05-05-2026)

[10] Sunshine et al. (2009). *Crying Wolf: An Empirical Study of SSL Warning Effectiveness* – Available at: https://www.usenix.org/legacy/event/sec09/tech/full_papers/sunshine.pdf (Last Accessed: 05-05-2026)

[11] Reeder et al. (2018). *An Experience Sampling Study of User Reactions to Browser Warnings in the Field* – Available at: https://www.researchgate.net/publication/324659530_An_Experience_Sampling_Study_of_User_Reactions_to_Browser_Warnings_in_the_Field (Last Accessed: 05-05-2026)

[12] Vance et al. (2017). *What Do We Really Know about How Habituation to Warnings Occurs Over Time? A Longitudinal fMRI Study of Habituation and Polymorphic Warnings* – Available at: https://dl.acm.org/doi/epdf/10.1145/3025453.3025896 (Last Accessed: 05-05-2026)

[13] Kirwan et al. (2020). *Repetition of Computer Security Warnings Results in Differential Repetition Suppression Effects as Revealed with Functional MRI* –

Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC7751389/pdf/fpsyg-11-528079.pdf (Last Accessed: 05-05-2026)

[14] Ofcom (2024). *Children and Parents: Media Use and Attitudes Report* – Available at: https://www.ofcom.org.uk/siteassets/resources/documents/research-and-data/media-literacy-research/children/children-media-use-and-attitudes-2024/childrens-media-literacy-report-2024.pdf (Last Accessed: 05-05-2026)

[15] Google (n.d.). *Making the world's information safely accessible* – Available at: https://safebrowsing.google.com (Last Accessed: 05-05-2026)

[16] Microsoft (n.d.). *Microsoft Defender SmartScreen* – Available at: https://learn.microsoft.com/en-us/windows/security/operating-system-security/virus-and-threat-protection/microsoft-defender-smartscreen/ (Last Accessed: 05-05-2026)

[17] Mozilla Support (2024). *How does built-in Phishing and Malware Protection Work?* – Available at: https://support.mozilla.org/en-US/kb/how-does-phishing-and-malware-protection-work (Last Accessed: 05-05-2026)

[18] Opera Help (n.d.). *Security and privacy* – Available at: https://help.opera.com/en/latest/security-and-privacy/ (Last Accessed: 05-05-2026)

[19] Figma (n.d.). *Login page* – Available at: https://www.figma.com/login?is_not_gen_0=true&resource_type=team (Last Accessed: 05-05-2026)

[20] Atlassian (n.d.). *What is Agile?* – Available at: https://www.atlassian.com/agile (Last Accessed: 05-05-2026)

[21] Trello (n.d.) *Stronghold* – Available at: https://trello.com/b/GL3SSZr6/stronghold (Last Accessed: 05-05-2026)

[22] W3 Schools (n.d.). *JavaScript Tutorial* – Available at: https://www.w3schools.com/js/default.asp (Last Accessed: 05-05-2026)

[23] ElectronJS (n.d.). *What is Electron?* – Available at: https://www.electronjs.org/docs/latest/ (Last Accessed: 05-05-2026)

[24] Visual Studio (n.d.). *Download Visual Studio Code* – Available at: https://code.visualstudio.com (Last Accessed: 05-05-2026)

[25] Firebase (n.d.). *About Firebase* – Available at: https://firebase.google.com (Last Accessed: 05-05-2026)

[26] W3 Schools (n.d.). *HTML Tutorial* – Available at: https://www.w3schools.com/html/default.asp (Last Accessed: 05-05-2026)

[27] W3 Schools (n.d.). *CSS Tutorial* – Available at: https://www.w3schools.com/css/default.asp (Last Accessed: 05-05-2026)

[28] AWS (n.d.). *What's the Difference Between HTTP and HTTPS?* – Available at: https://aws.amazon.com/compare/the-difference-between-https-and-http/ (Last Accessed: 05-05-2026)

[29] eSecurity Planet (2011). *Detecting Malicious Traffic in HTTP Headers* – Available at: https://www.esecurityplanet.com/trends/detecting-malicious-traffic-in-http-headers/ (Last Accessed: 05-05-2026)

[30] WhoIs (n.d.). *Domain Name Information Lookup* – Available at: https://who.is (Last Accessed: 05-05-2026)

[31] OWASP (n.d.). *HTTP Security Response Headers Cheat Sheet* – Available at: https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html (Last Accessed: 05-05-2026)

[32] Yomguithereal (2021). *Talisman Library* – Available at: https://github.com/yomguithereal/talisman (Last Accessed: 05-05-2026)

[33] npm (2020). *fast-levenshtein – Levenshtein algorithm in Javascript* – Available at: https://www.npmjs.com/package/fast-levenshtein (Last Accessed: 05-05-2026)

[34] npm (2019). *Js-levenshtein* – Available at: https://www.npmjs.com/package/js-levenshtein (Last Accessed: 05-05-2026)

[35] Malwarebytes (n.d.). *Cookie hijacking: how to protect your personal information from online attacks* – Available at: https://www.malwarebytes.com/cybersecurity/basics/cookie-hijacking (Last Accessed: 05-05-2026)

[36] Kaseya (2025). *What is token theft?* – Available at: https://www.kaseya.com/blog/what-is-token-theft/ (Last Accessed: 05-05-2026)

[37] Statista (2026). *Most common malicious file types received globally via web and e-mail in 2024* – Available at: https://www.statista.com/statistics/1238996/top-malware-by-file-type/?srsltid=AfmBOooNzvDryrTxml0TCvJLKlRaA-R1qx7Jf8TW4pzNbdiBQKQy6VKS#google_vignette (Last Accessed: 05-05-2026)

[38] Microsoft (n.d.). *Download Microsoft 365* – Available at: https://www.microsoft.com/en-us/microsoft-365/download-office (Last Accessed: 05-05-2026)

[39] AWS (n.d.). *What is an SSL/TLS Certificate?* – Available at: https://aws.amazon.com/what-is/ssl-certificate/ (Last Accessed: 05-05-2026)

[40] Google Identity (n.d.). *Using OAuth 2.0 to Access Google APIs* – Available at: https://developers.google.com/identity/protocols/oauth2 (Last Accessed: 05-05-2026)

[41] Bitwarden (n.d.). *What is password hashing?* – Available at: https://bitwarden.com/resources/what-is-password-hashing/ (Last Accessed: 05-05-2026)

[42] GeeksforGeeks (2025). *What Is Password Salting and How Does It Work?* – Available at: https://www.geeksforgeeks.org/techtips/what-is-password-salting/ (Last Accessed: 05-05-2026)

[43] Electron (2021). *LICENSE* – Available at: https://github.com/electron/electron/blob/main/LICENSE (Last Accessed: 05-05-2026)

[44] Yomguithereal (2020). *LICENSE.txt* – Available at: https://github.com/Yomguithereal/talisman/blob/master/LICENSE.txt (Last Accessed: 05-05-2026)

[45] npm (2019). *whois-json* – Available at: https://www.npmjs.com/package/whois-json (Last Accessed: 05-05-2026)

[46] Firebase (n.d.). *Pricing plans* – Available at: https://firebase.google.com/pricing (Last Accessed: 05-05-2026)

[47] GOV.UK (n.d.). *The UK's data protection legislation* – Available at: https://www.gov.uk/data-protection (Last Accessed: 05-05-2026)

[48] Microsoft (2022). *Five tips for building inclusive and accessible software* – Available at: https://www.microsoft.com/en-us/startups/blog/inclusive-and-accessible-software/ (Last Accessed: 05-05-2026)

[49] GitHub Desktop (n.d.). *Download GitHub Desktop* – Available at: https://desktop.github.com/download/ (Last Accessed: 05-05-2026)

[50] Google Forms (n.d.). *COMP3000 User Acceptance Testing* – Available at: https://forms.gle/aBaTYnZtvrMqfijt7 (Last Accessed: 05-05-2026)

[51] Cloudflare (n.d.). *What is DNS? | How DNS works* – Available at: https://www.cloudflare.com/en-gb/learning/dns/what-is-dns/ (Last Accessed: 05-05-2026)

[52] Google Cloud (n.d.). *Gemini API* – Available at: https://console.cloud.google.com/apis/library/generativelanguage.googleapis.com?project=stronghold-e0ef2 (Last Accessed: 05-05-2026)

12 Appendices

12.1 Appendix A – Firebase Screenshots

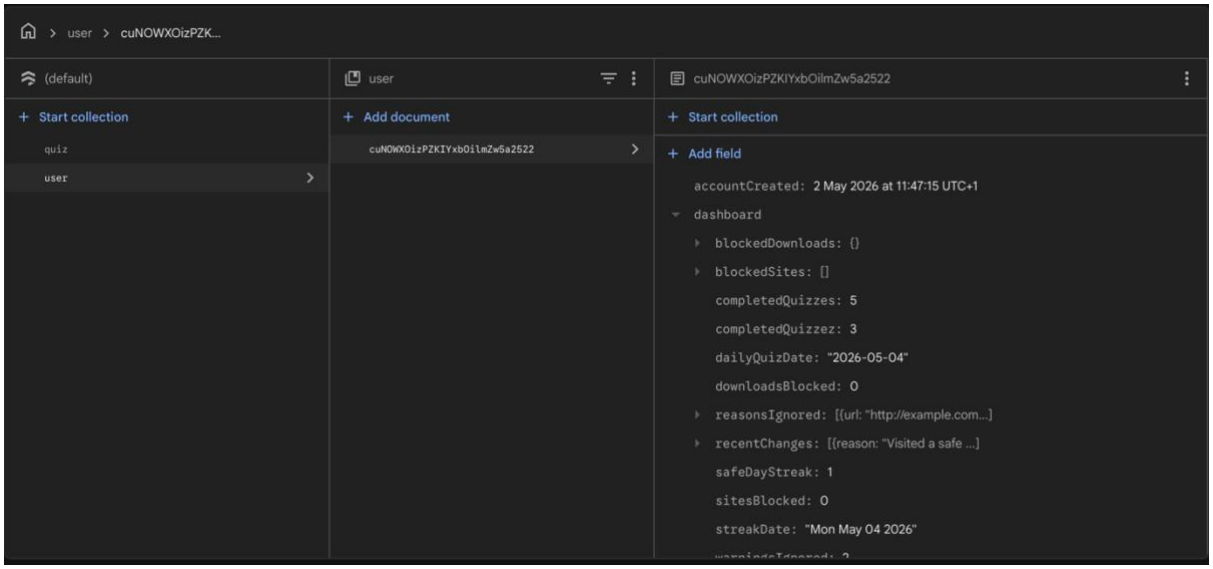

Figure 19 – Firebase, Example Document for a Test User

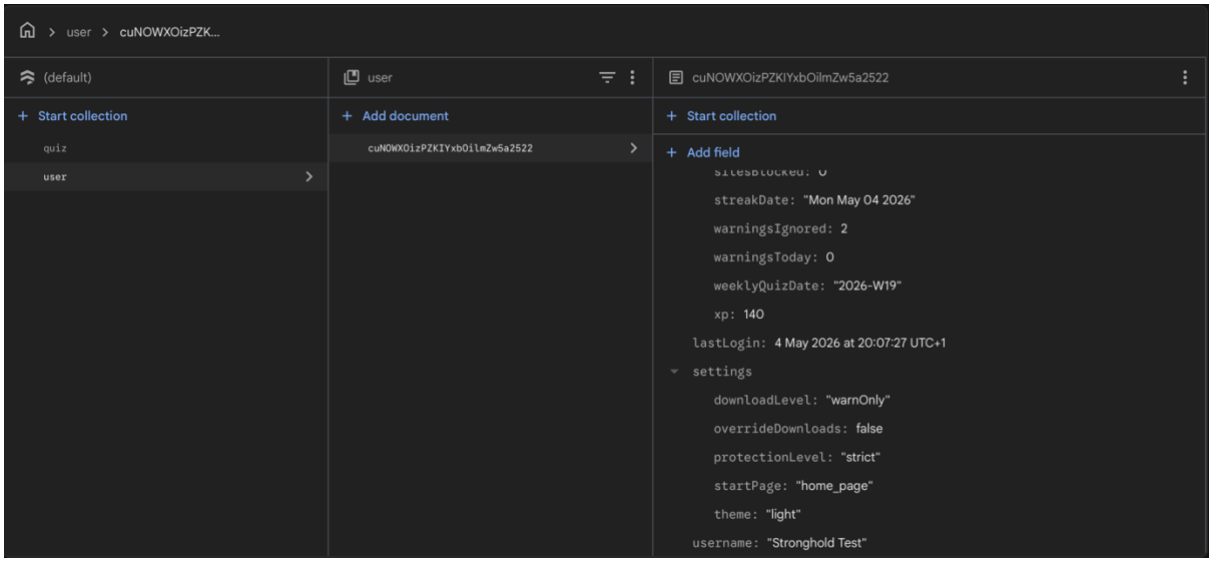

Figure 20 – Firebase, Example Document for a Test User (cont.)

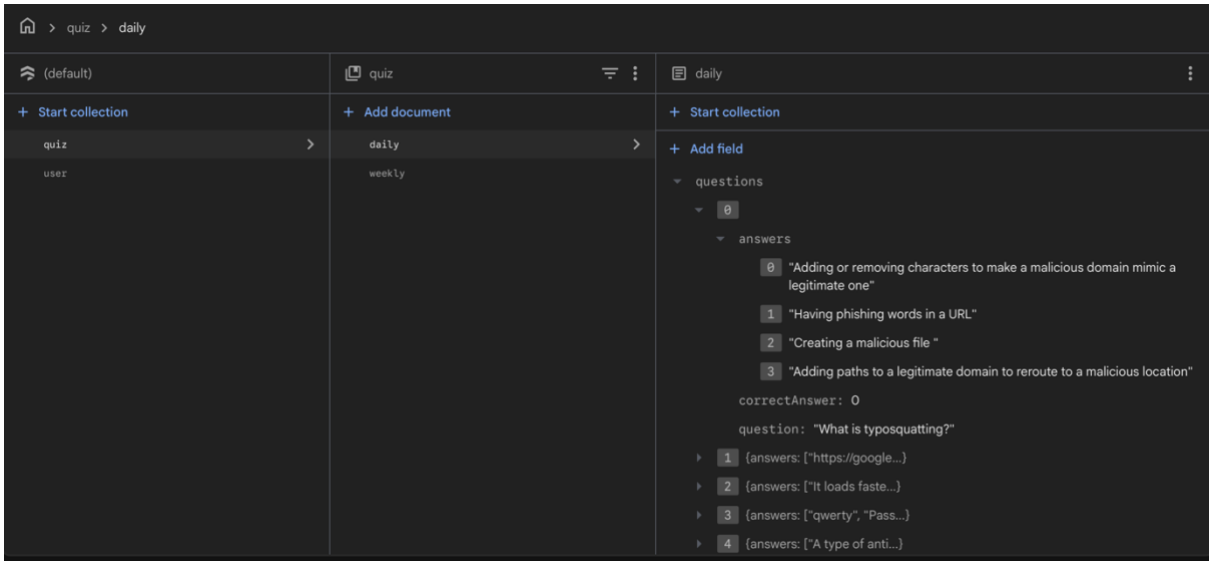

Figure 21 – Firebase, Example Document for Daily Quiz

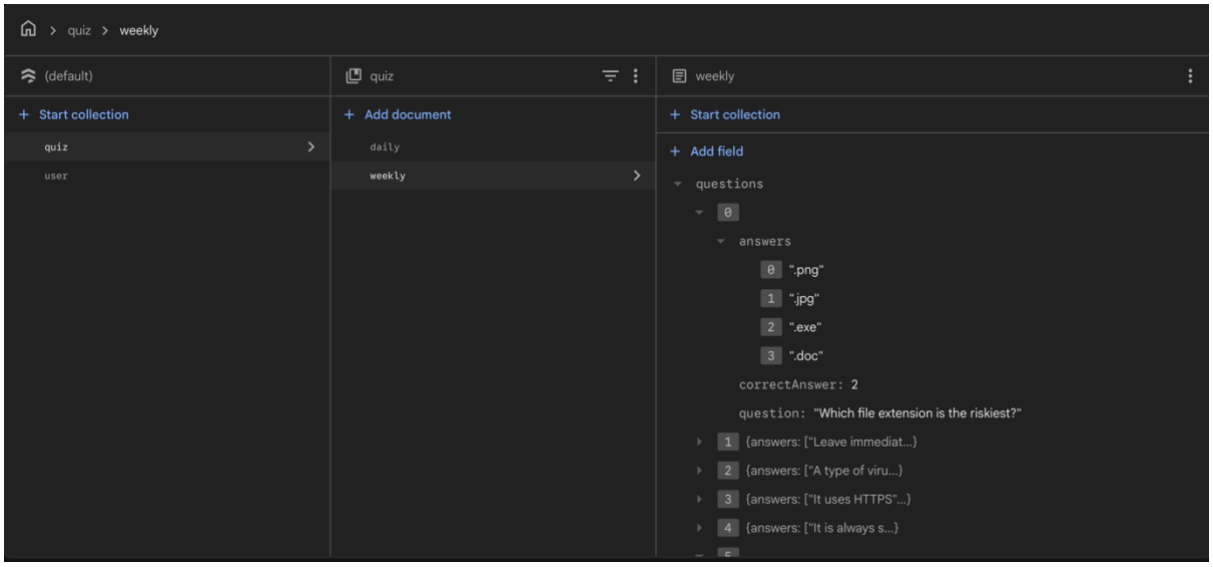

Figure 22 –Firebase, Example Document for Weekly Quiz

12.2 Appendix B – Trello Screenshots

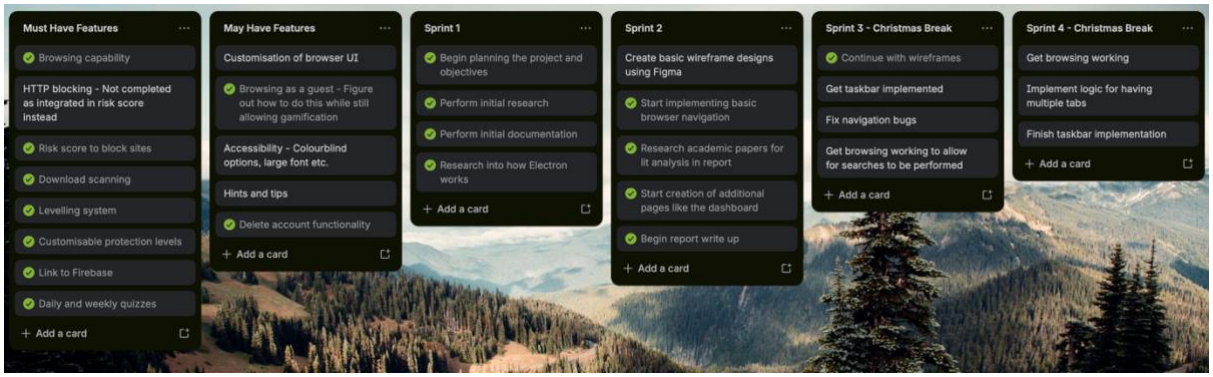

Figure 23 – Trello, Must Have and May Have Features, and Sprints 1 to 4

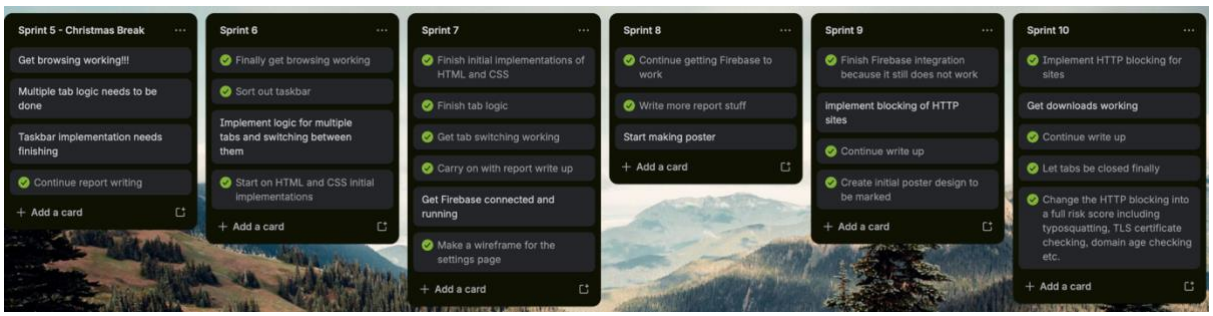

Figure 24 – Trello, Sprints 5 to 10

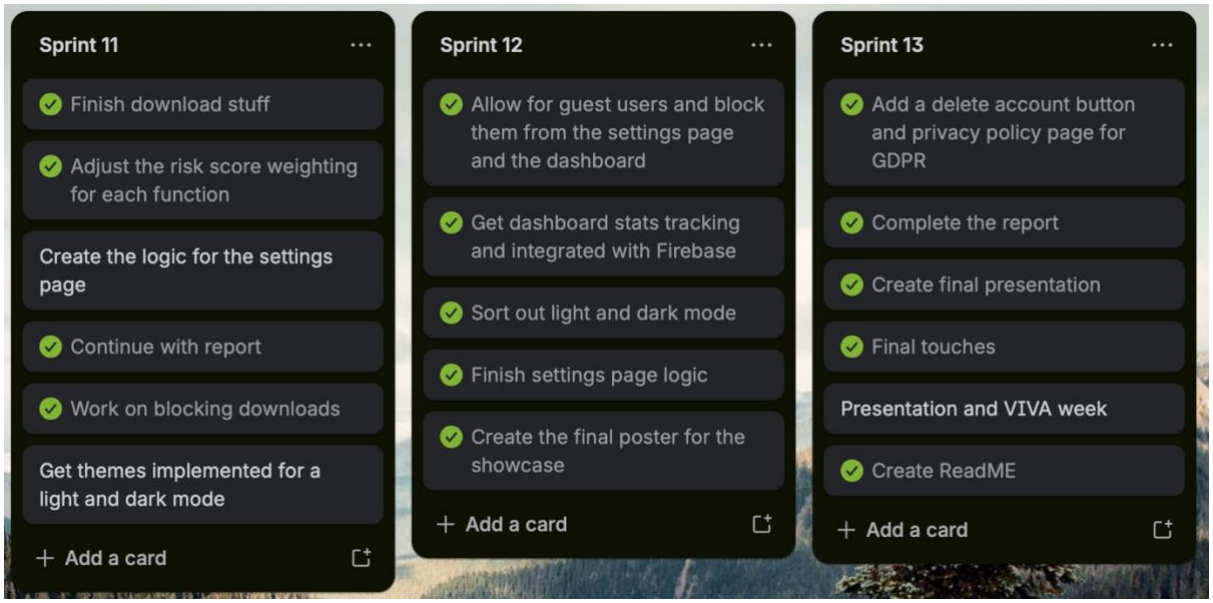

Figure 25 – Trello, Sprints 11 to 13

12.3 Appendix C – Final UI Implementation Screenshots

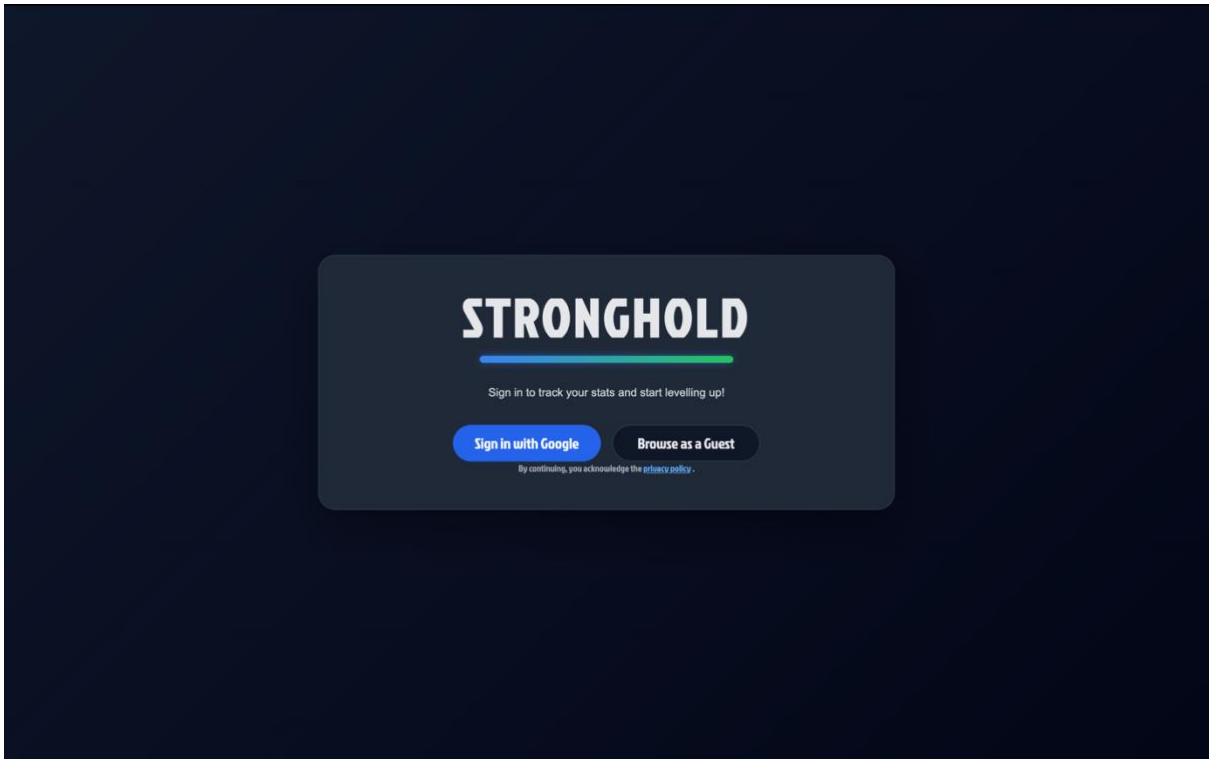

Figure 26 – Stronghold, Final Start Page UI

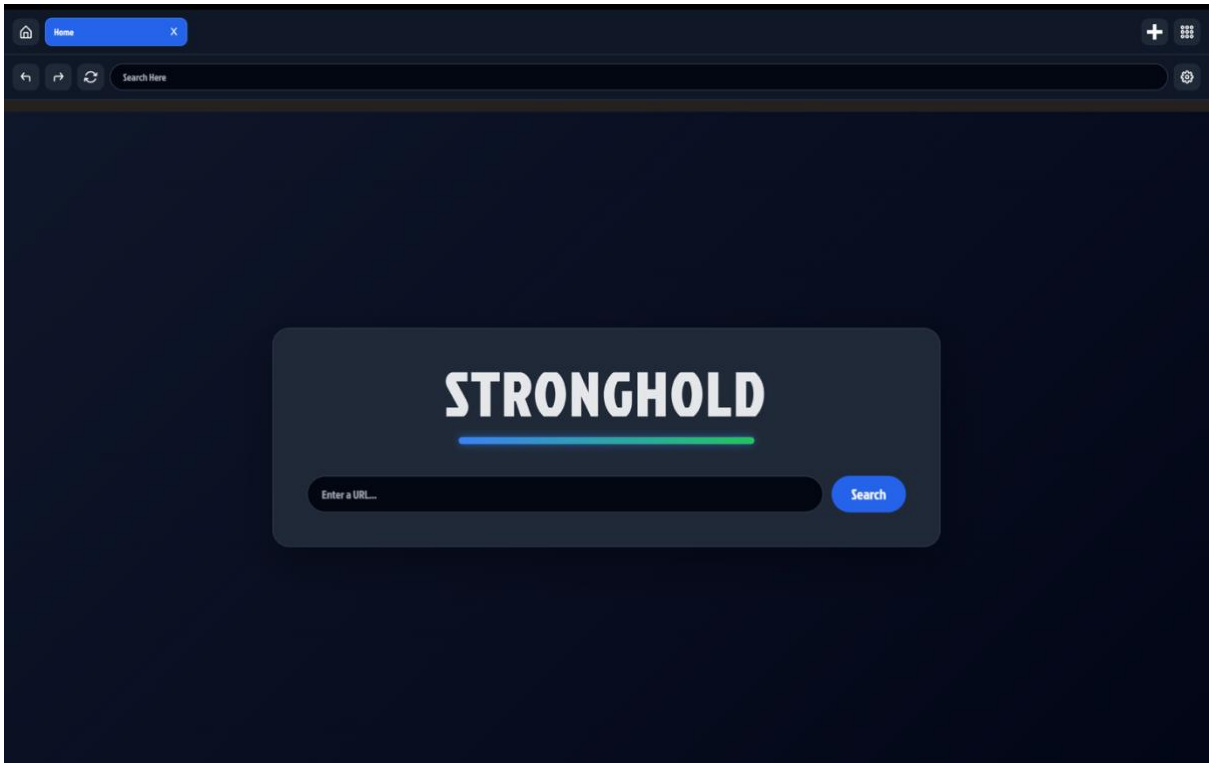

Figure 27 – Stronghold, Final Home Page UI

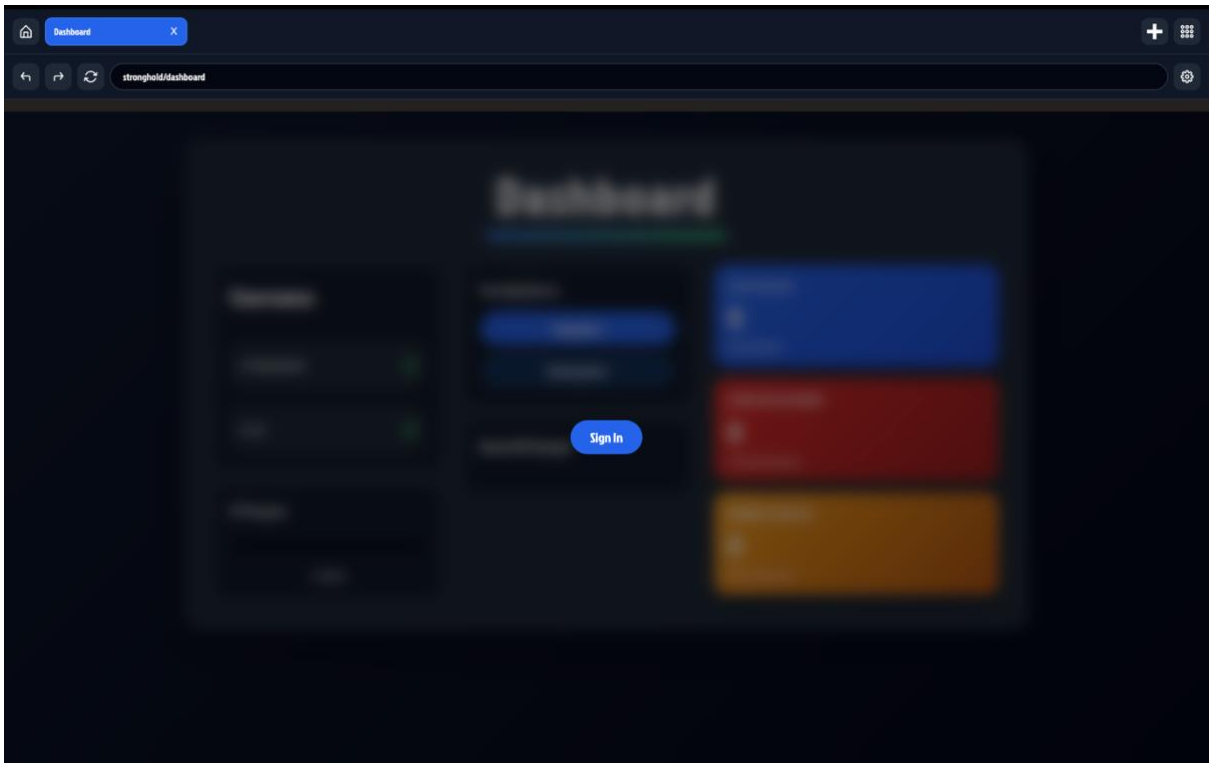

Figure 28 – Stronghold, Final Guest Browsing Dashboard View

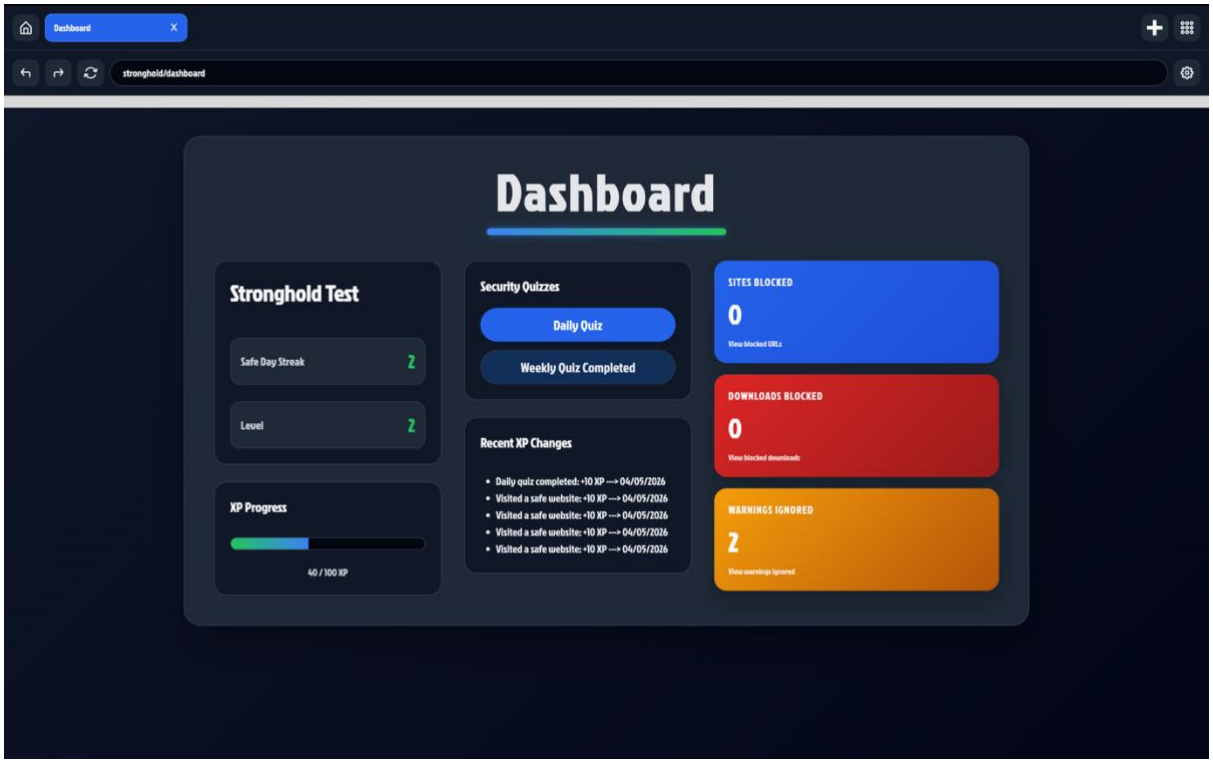

Figure 29 – Stronghold, Final Dashboard UI

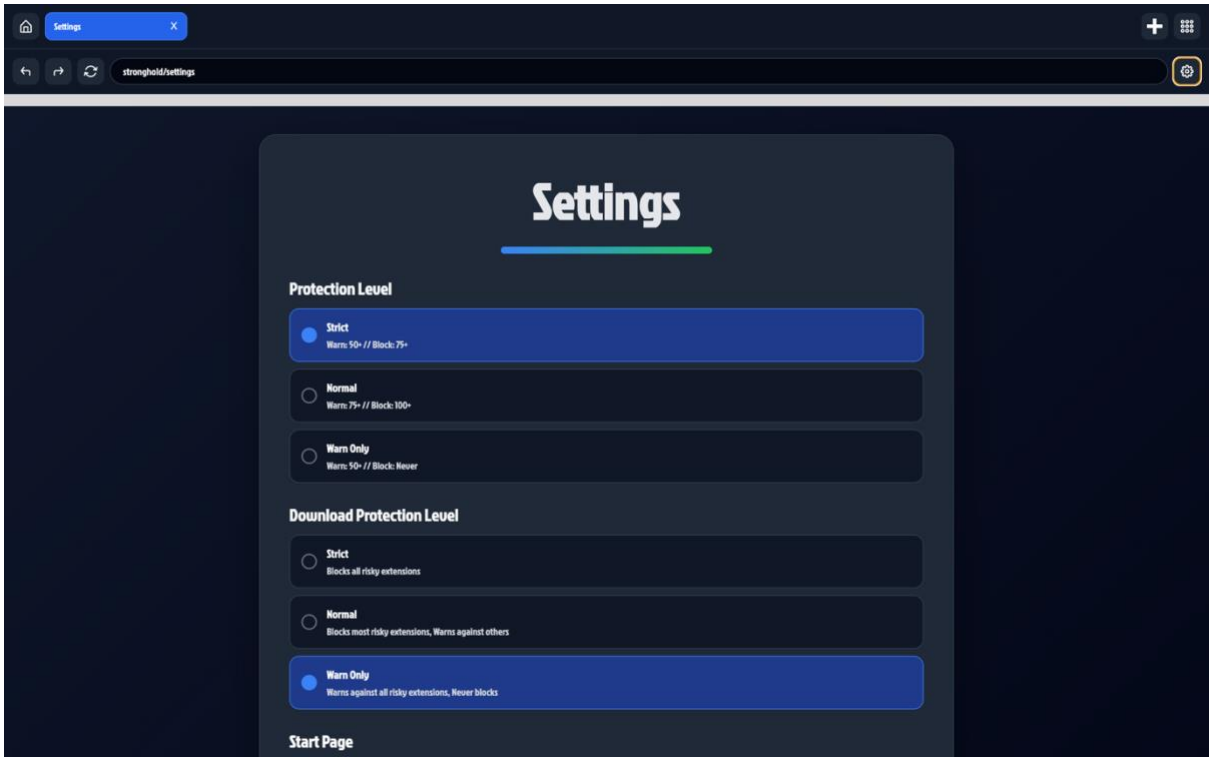

Figure 30 – Stronghold, Final Settings Page UI

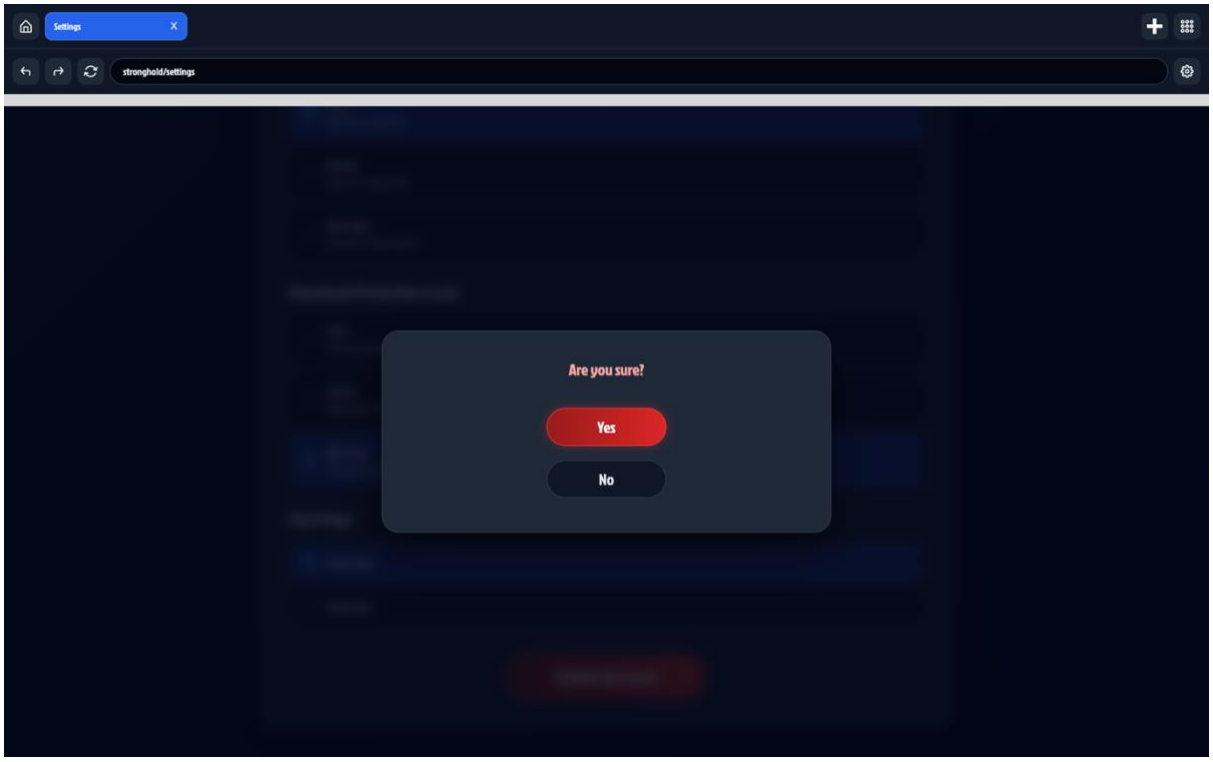

Figure 31 – Stronghold, Final Delete Account Confirmation

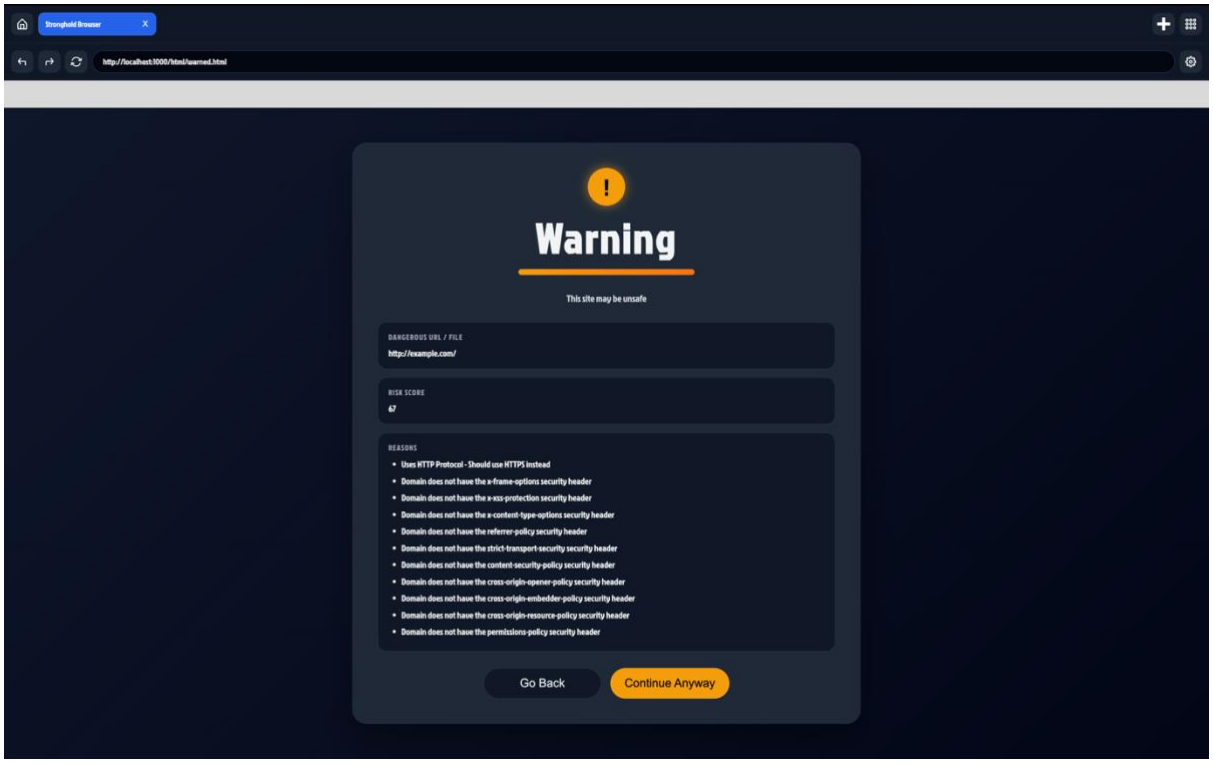

Figure 32 – Stronghold, Final URL Warning UI

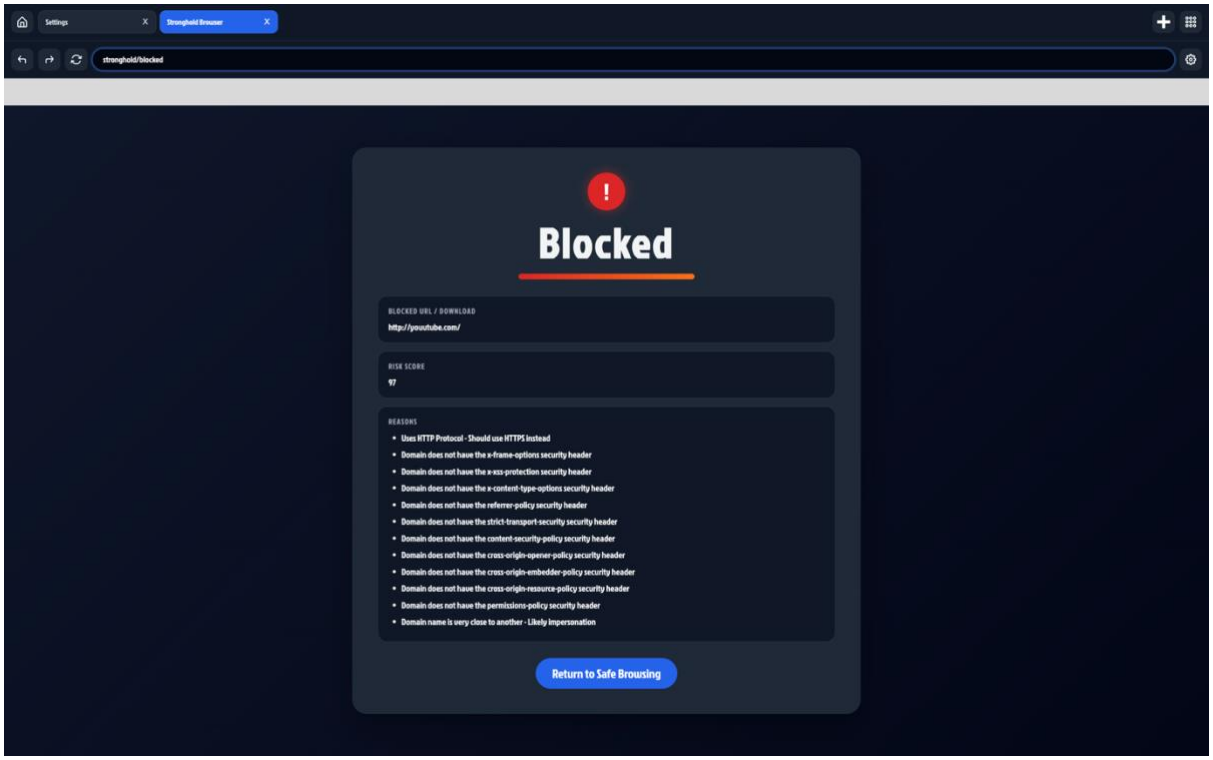

Figure 33 – Stronghold, Final URL Blocking UI

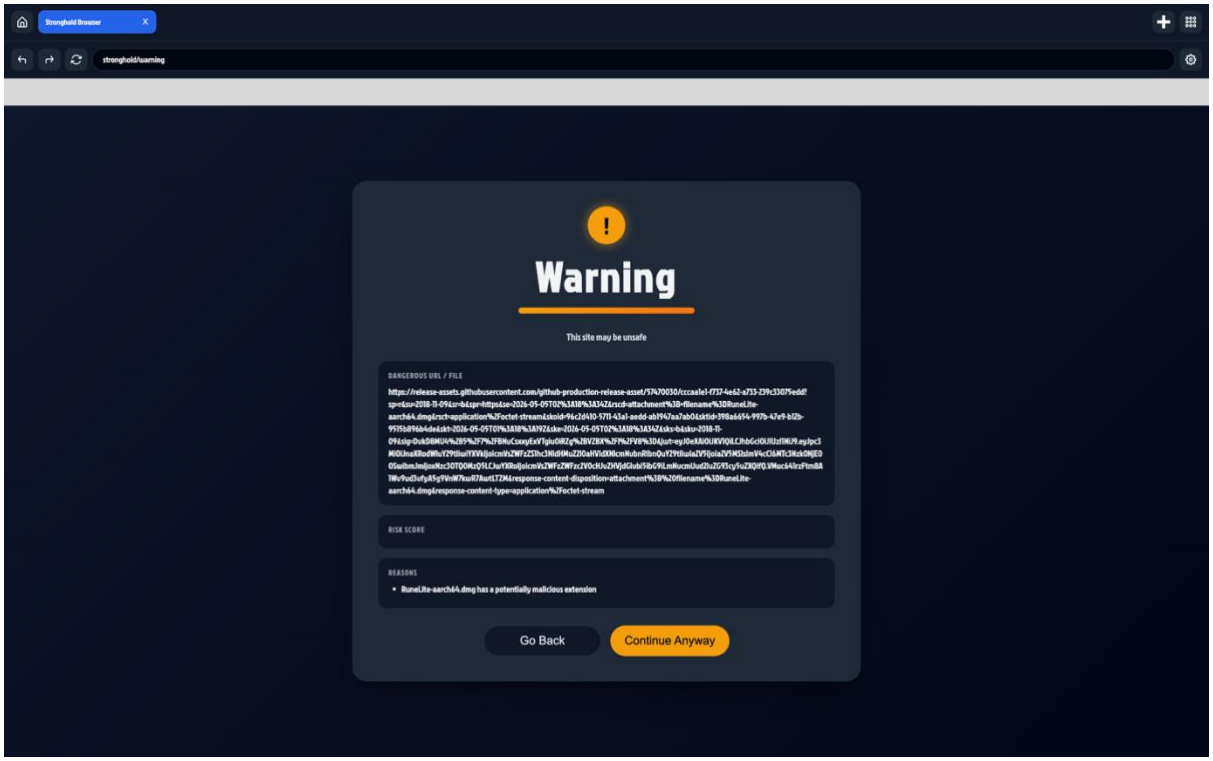

Figure 34 – Stronghold, Final Download Warning UI

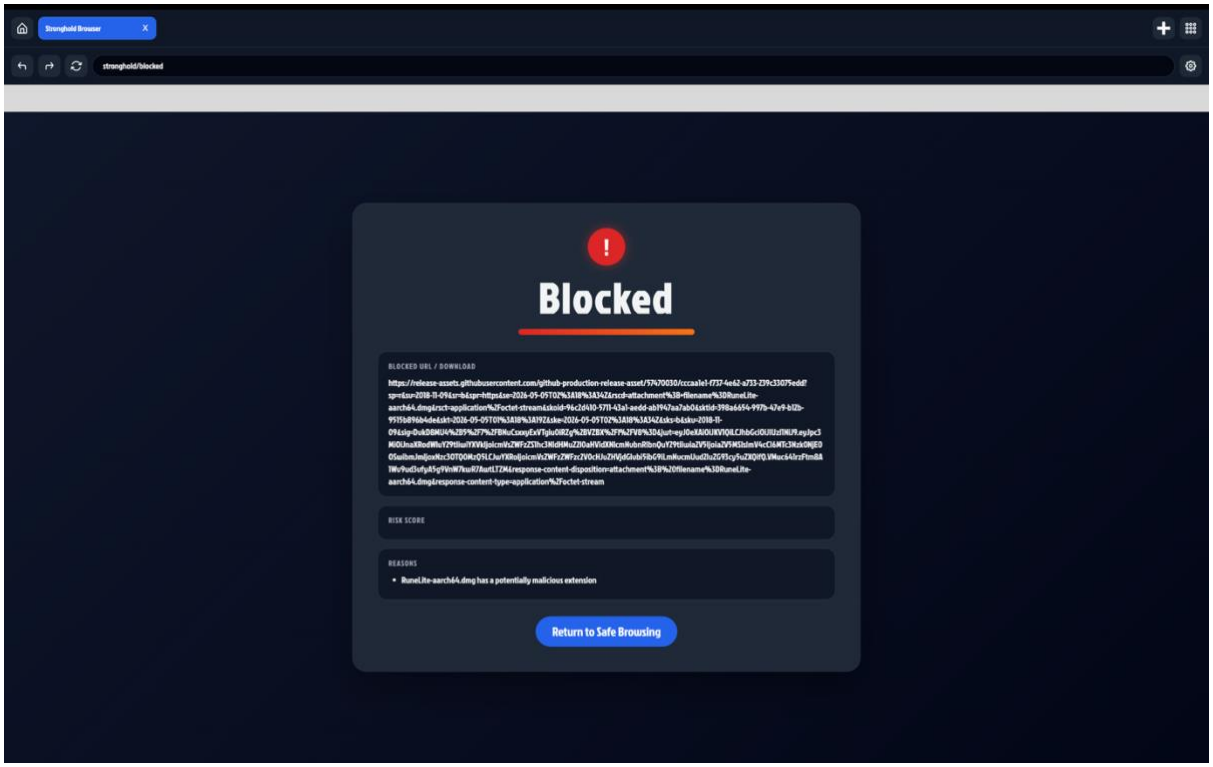

Figure 35 – Stronghold, Final Download Blocking UI

12.4 Appendix D – Google Form Screenshots

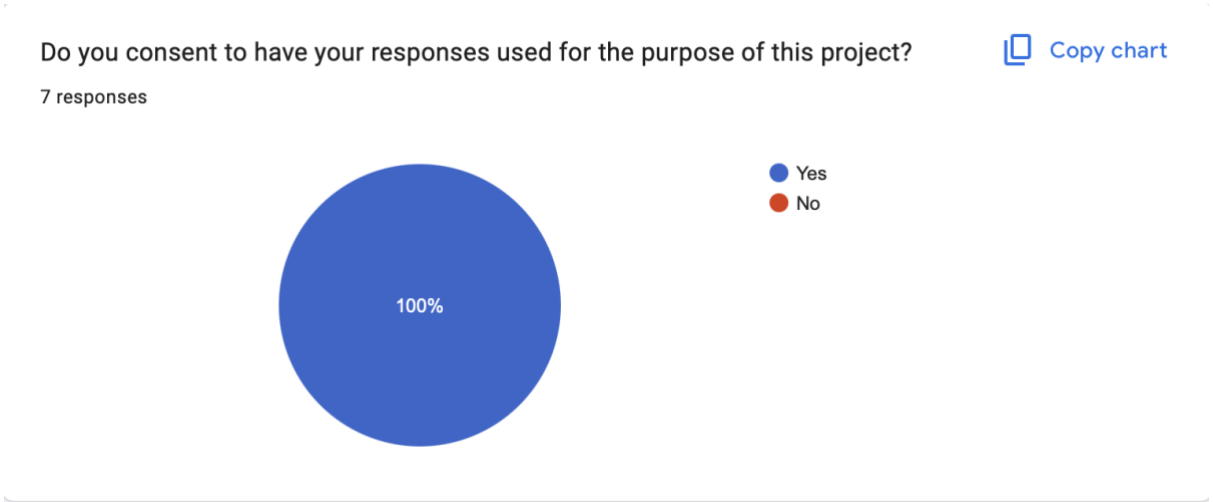

Figure 36 – Testing, Consent Question

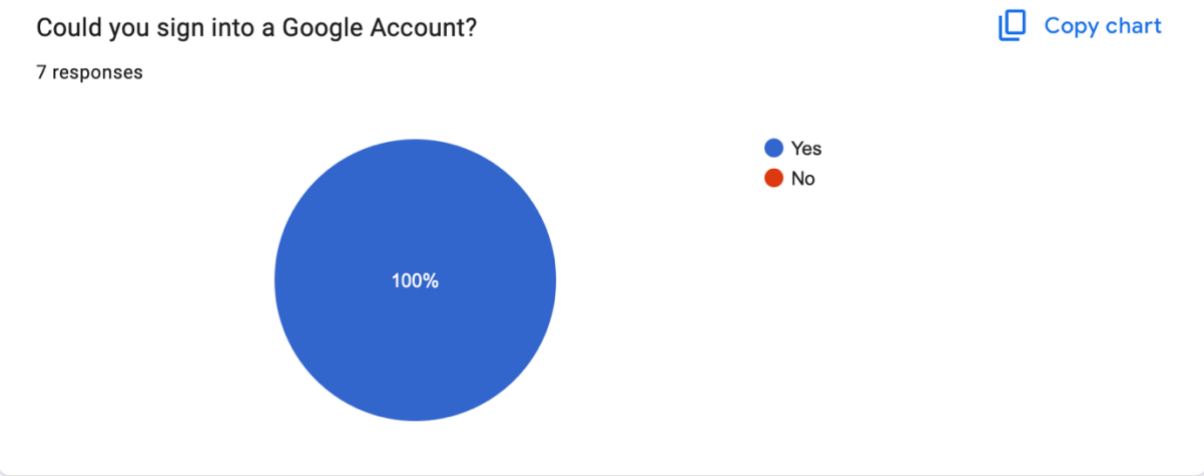

Figure 37 – Testing, Task 1 Results

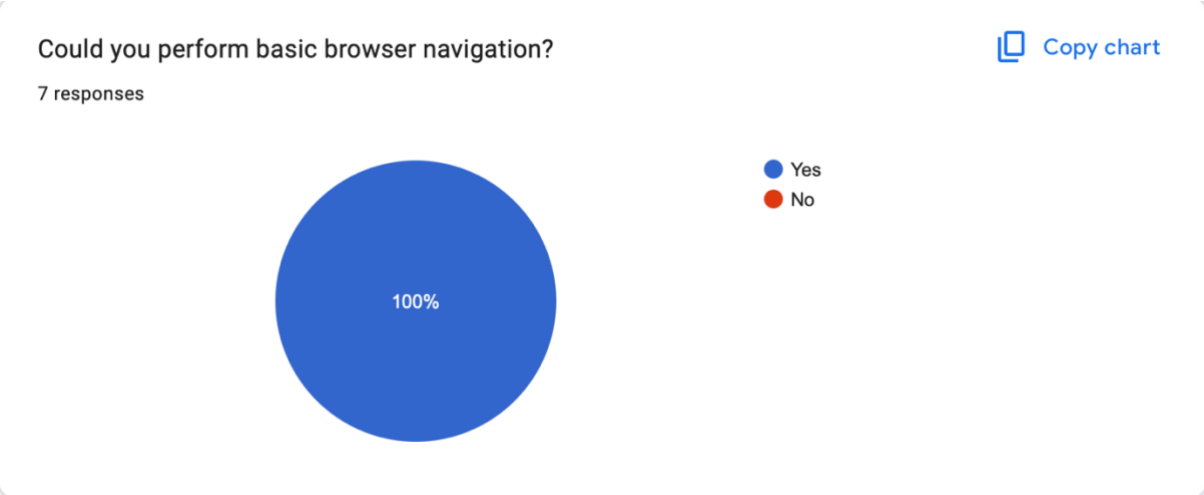

Figure 38 – Testing, Task 2 Results

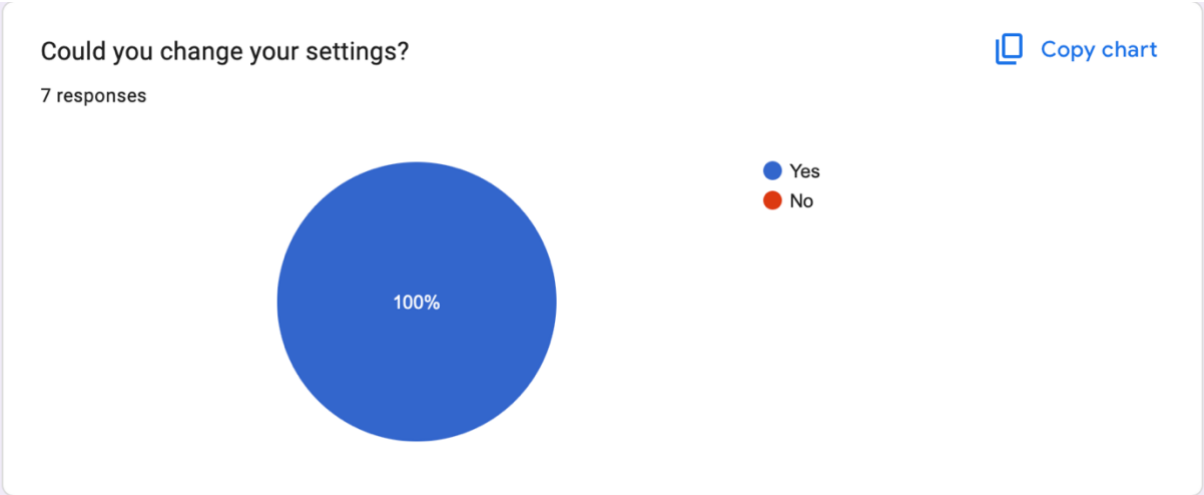

Figure 39 – Testing, Task 3 Results

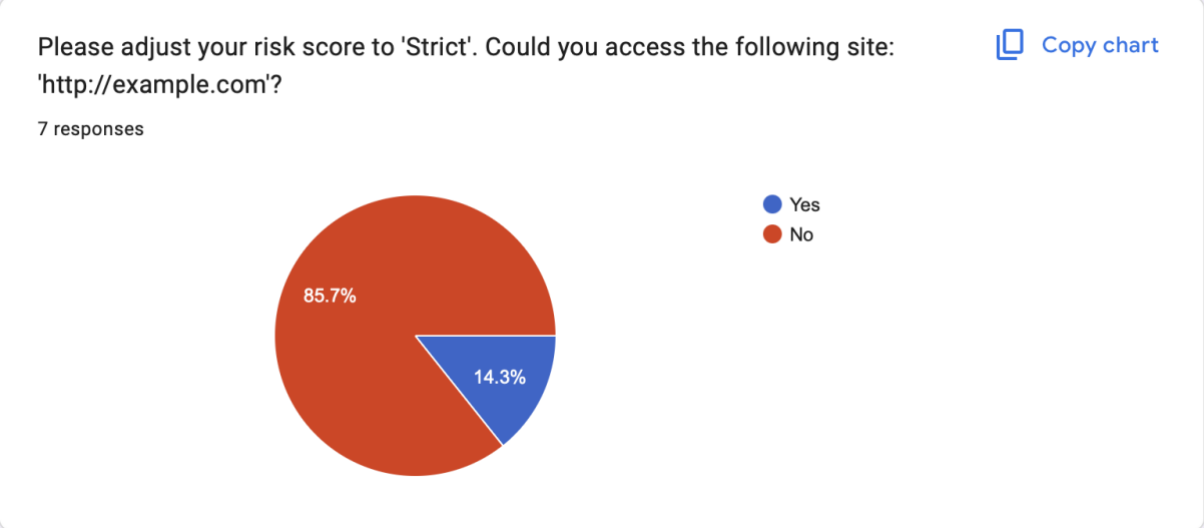

Figure 40 – Testing, Task 4 Results

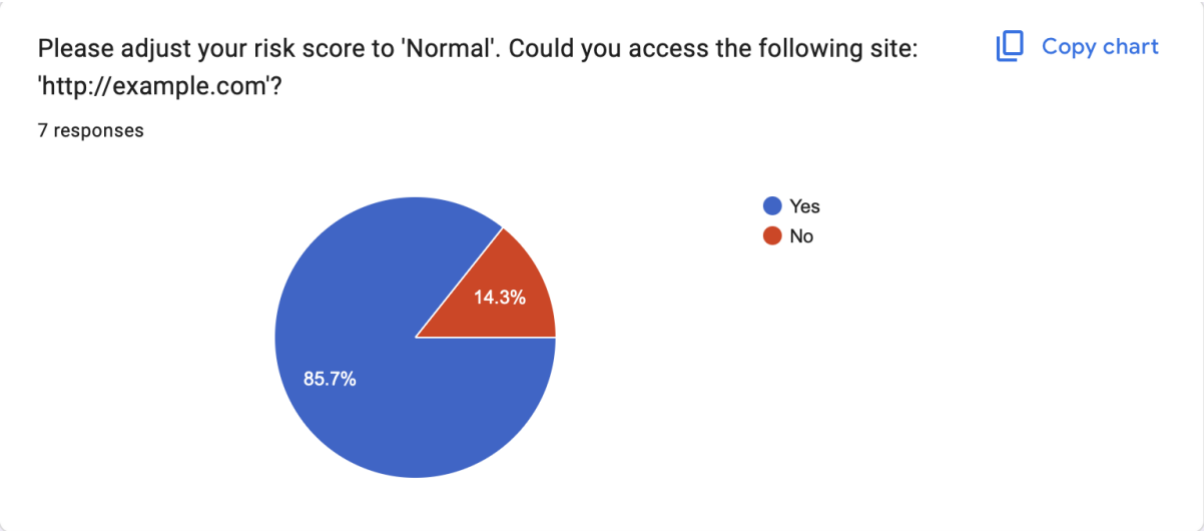

Figure 41 – Testing, Task 5 Results

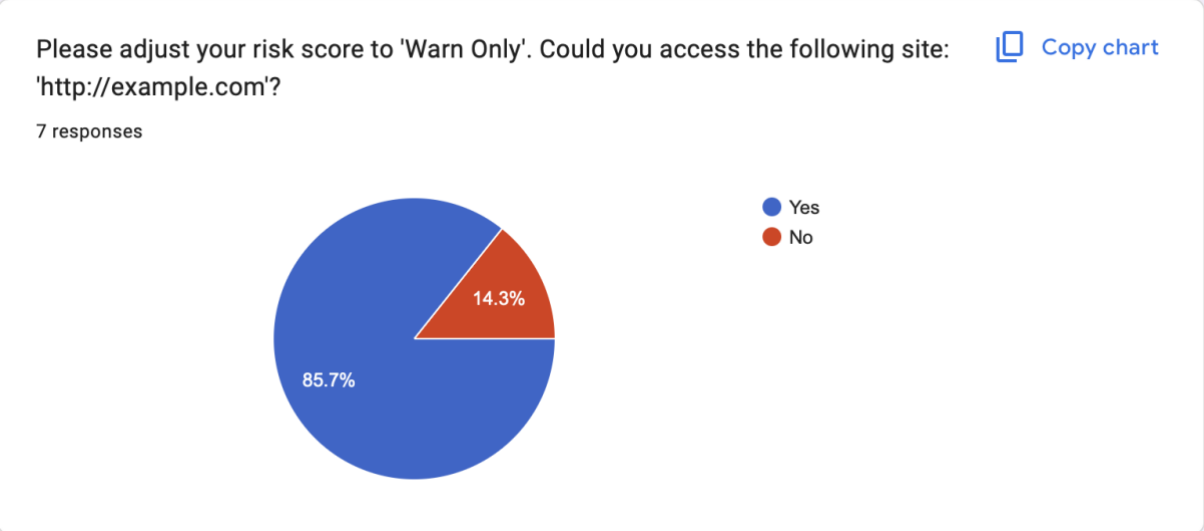

Figure 42 – Testing, Task 6 Results

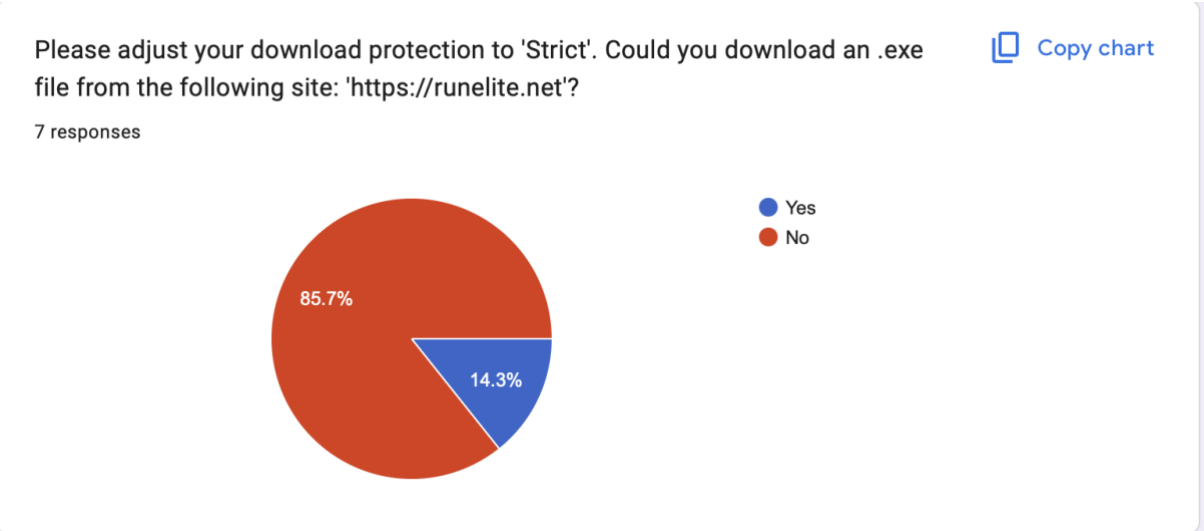

Figure 43 – Testing, Task 7 Results

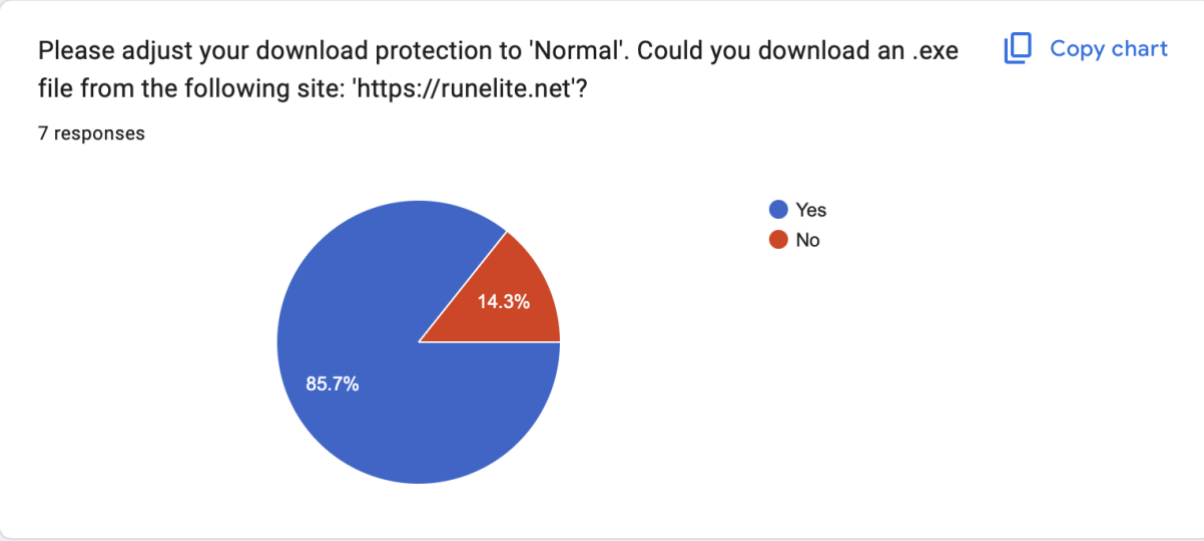

Figure 44 – Testing, Task 8 Results

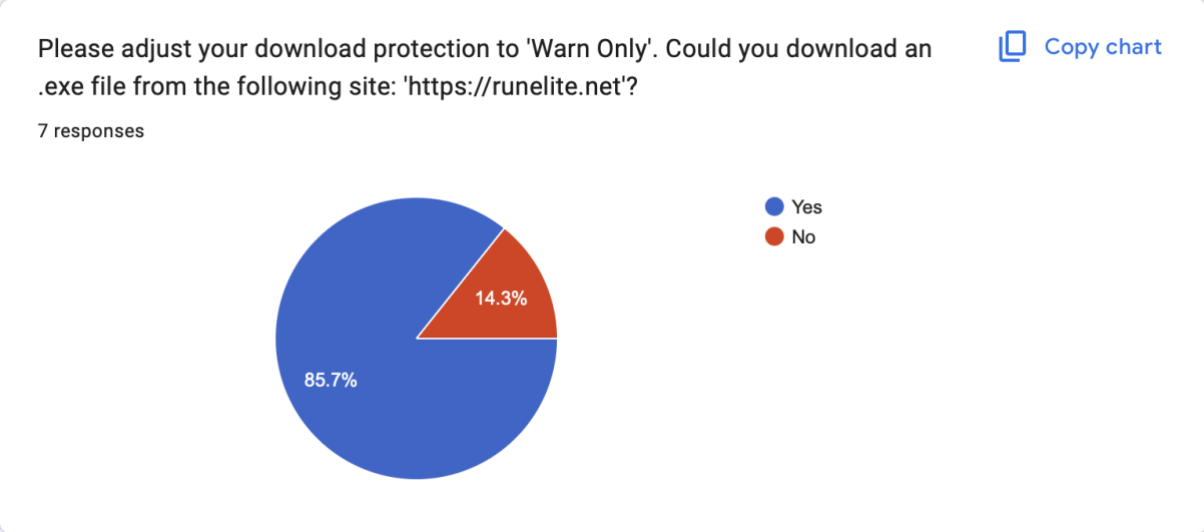

Figure 45 – Testing, Task 9 Results

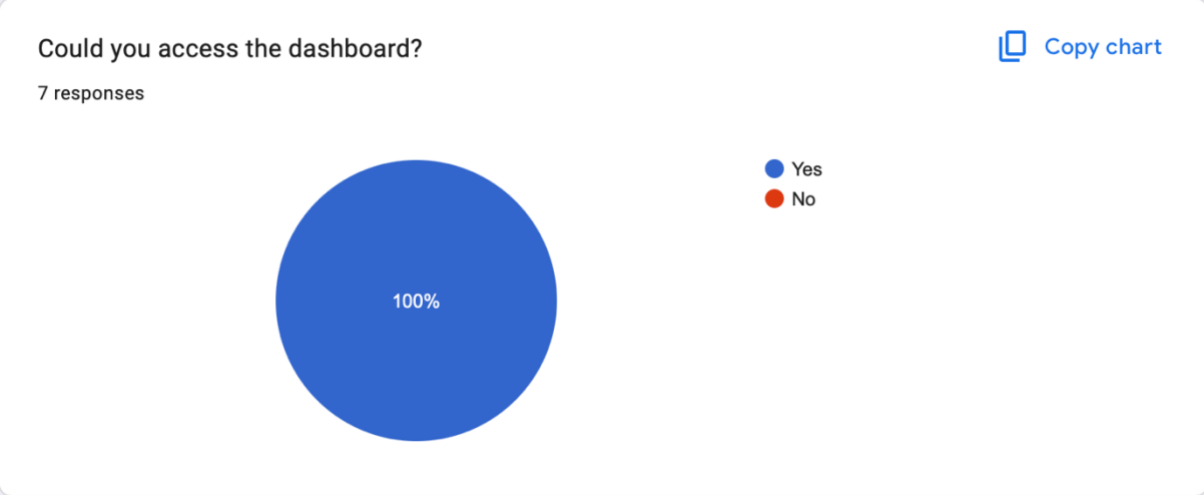

Figure 46 – Testing, Task 10 Results

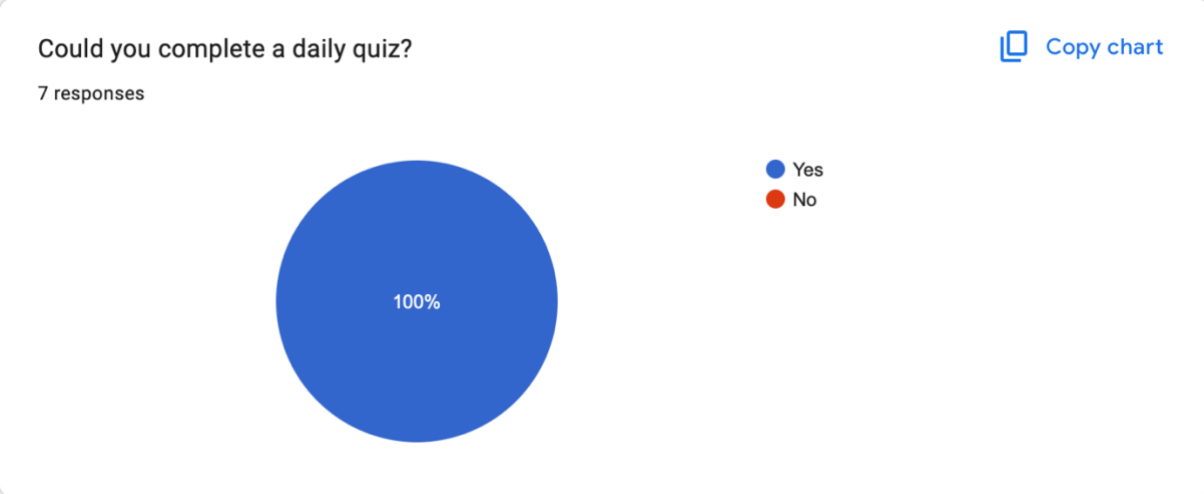

Figure 47 – Testing, Task 11 Results

Figure 48 – Testing, Task 12 Results

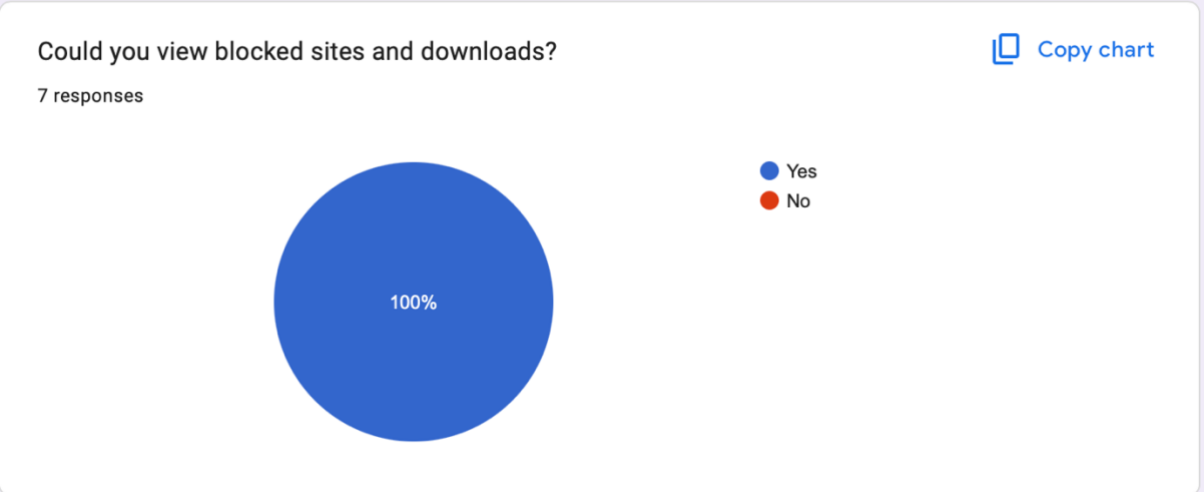

Figure 49 – Testing, Task 13 Results

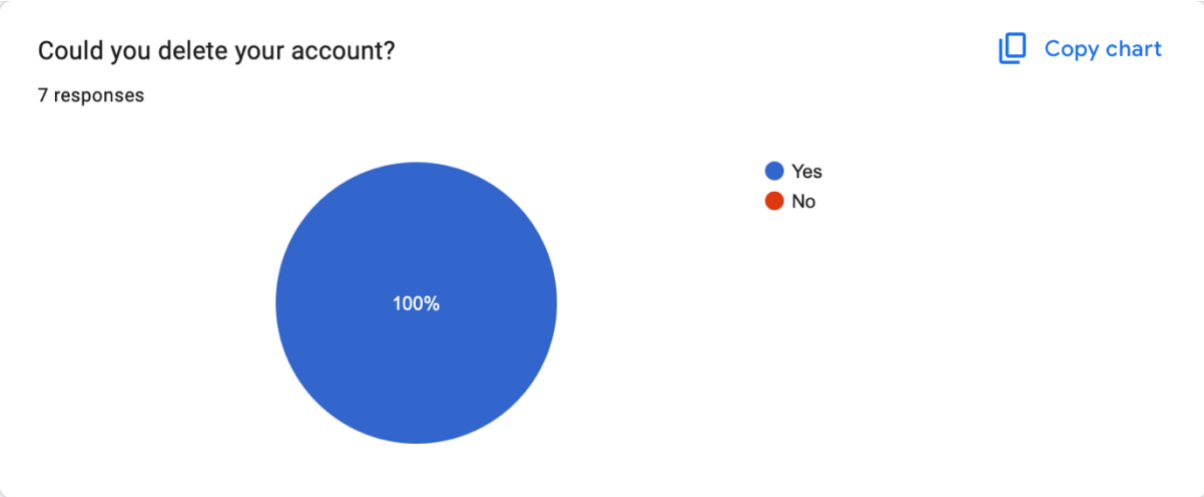

Figure 50 – Testing, Task 14 Results

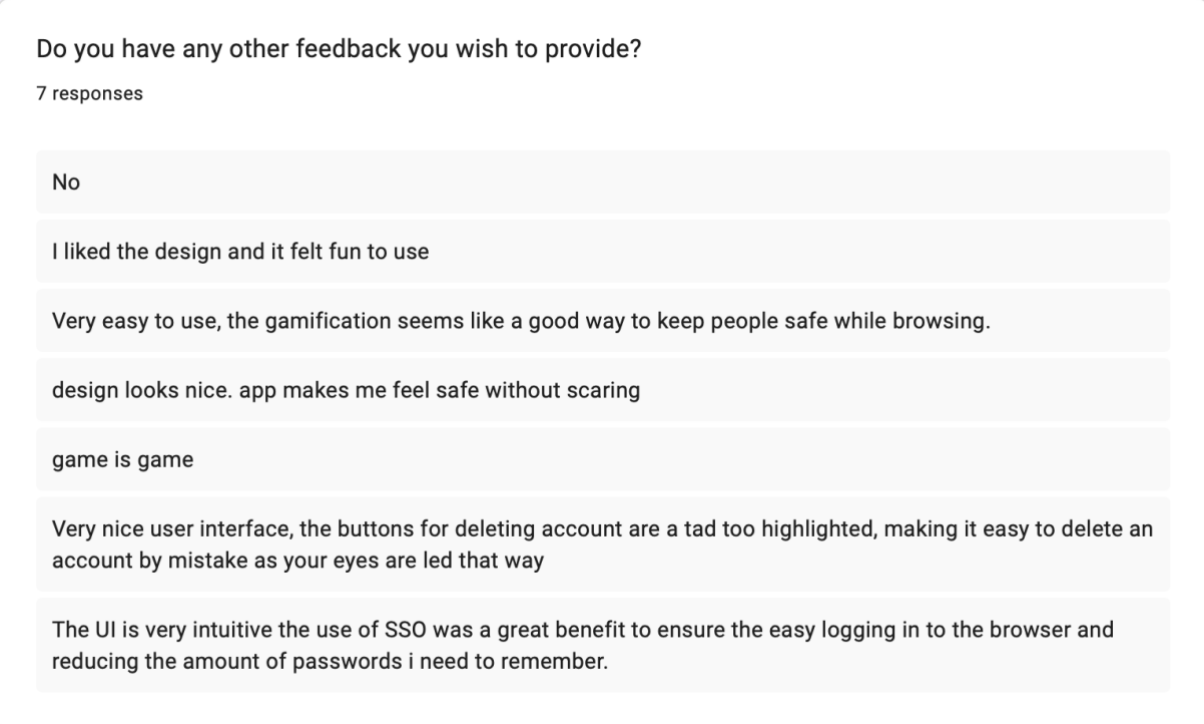

Figure 51 – Testing, Final Feedback Results

12.5 AI Declaration

Solo Work	**S1 - Generative AI tools have not been used for this assessment.**	☐
Assisted Work	**A1 – Idea Generation and Problem Exploration** Used to generate project ideas, explore different approaches to solving a problem, or suggest features for software or systems. Students must critically assess AI-generated suggestions and ensure their own intellectual contributions are central.	☐
	A2 - Planning & Structuring Projects AI may help outline the structure of reports, documentation and projects. The final structure and implementation must be the student's own work.	☒
	A3 – Code Architecture AI tools maybe used to help outline code architecture (e.g. suggesting class hierarchies or module breakdowns). The final code structure must be the student's own work.	☐
	A4 – Research Assistance Used to locate and summarise relevant articles, academic papers, technical documentation, or online resources (e.g. Stack Overflow, GitHub discussions. The interpretation and integration of research into the assignment remain the student's responsibility.	☐
	A5 - Language Refinement Used to check grammar, refine language, improve sentence structure in documentation not code. AI should be used only to provide suggestions for improvement. Students must ensure that the documentation accurately reflects the code and is technically correct.	☒
	A6 – Code Review AI tools can be used to check comments within the code and to suggest improvements to code readability, structure or syntax. AI should be used only to provide suggestions for improvement. Students must ensure that the code accurately reflects their knowledge and is technically correct.	☒
	A7 - Code Generation for Learning Purposes Used to generate example code snippets to understand syntax, explore alternative implementations, or learn new programming paradigms. Students must not submit AI-generated code as their own and must be able to explain how it works.	☒
	A8 - Technical Guidance & Debugging Support AI tools can be used to explain algorithms, programming concepts, or debugging strategies. Students may also help interpret error messages or suggest possible fixes. However, students must write, test, and debug their own code independently and understand all solutions submitted.	☒
	A9 - Testing and Validation Support AI may assist in generating test cases, validating outputs, or suggesting edge cases for software testing. Students are responsible for designing comprehensive test plans and interpreting test results.	☐
	A10 - Data Analysis and Visualization Guidance AI tools can help suggest ways to analyse datasets or visualize results (e.g.	☐

	recommending chart types or statistical methods). Students must perform the analysis themselves and understand the implications of the results.	
	A11 - Other uses not listed above Please specify:	☐
Partnered Work	**P1 - Generative AI tool usage has been used integrally for this assessment** Students can adopt approaches that are compliant with instructions in the assessment brief. Please Specify:	☐

Please provide details of AI usage and which elements of the coursework this relates to: **A2 – Used to help structure this report** **A5 – Used to refine grammar and language in this report** **A6 – Used to improve code structure** **A7 – Used to generate pseudocode to assist in the final development** **A8 – Used to explain some algorithms**

I understand that the ownership and responsibility for the academic integrity of this submitted assessment falls with me, the student.	☒
I confirm that all details provide above are an accurate description of how AI was used for this assessment.	☒